

Chapter 04. SQL 활용

4-1. 표준 조인(STANDARD JOIN)

➤ 일반 집합 연산자와 SQL의 비교

일반 집합 연산자	SQL문	설명
UNION 연산	UNION 기능으로 구현	• UNION 연산은 수학적 합집합을 제공하기 위해, 공통 교집합의 중복을 없애기 위한 사전 작업으로 시스템에 부하를 주는 정렬 작업이 발생한다. • 이후 UNION ALL 기능이 추가되었는데, 특별한 요구 사항이 없다면 공통 집합을 중복해서 그대로 보여 주기 때문에 정렬 작업이 일어나지 않는 장점을 가진다. • 만일 UNION과 UNION ALL의 출력 결과가 같다면 응답 속도 향상이나 자원 효율화 측면에서 데이터 정렬 작업이 발생하지 않는 UNION ALL을 사용하는 것을 권고한다.
INTERSECTION 연산	INTERSECT 기능으로 구현	• INTERSECTION은 수학의 교집합으로써 두 집합의 공통 집합을 추출한다.
DIFFERENCE 연산	EXCEPT(Oracle은 MINUS) 기능으로 구현	• DIFFERENCE는 수학의 차집합으로써 첫 번째 집합에서 두 번째 집합과의 공통 집합을 제외한 부분이다. • 대다수 벤더는 EXCEPT를 Oracle은 MINUS 용어를 사용한다.
PRODUCT 연산	CROSS JOIN 기능으로 구현	• PRODUCT의 경우는 CROSS(ANIS/ISO 표준) PRODUCT라고 불리는 곱집합으로 JOIN 조건이 없는 경우 생길 수 있는 모든 데이터의 조합을 말한다. • 양쪽 집합의 M*N 건의 데이터 조합이 발생하며, CARTESIAN(수학자 이름) PRODUCT라고도 표현한다.

➤ 순수 관계 연산자와 SQL의 비교

순수 관계 연산자	SQL문	설명
SELECT 연산	WHERE 절로 구현	• SELECT 연산은 SQL 문장에서는 WHERE 절 기능으로 구현이 되었다.
PROJECT 연산	SELECT 절로 구현	• PROJECT 연산은 SQL 문장에서는 SELECT 절의 컬럼 선택 기능으로 구현되었다.
(NATURAL) JOIN 연산	다양한 JOIN 기능으로 구현	• JOIN 연산은 WHERE 절의 INNER JOIN 조건과 함께 FROM 절의 NATURAL JOIN, INNER JOIN, OUTER JOIN, USING 조건절, ON 조건절 등으로 가장 다양하게 발전하였다.
DIVIDE 연산	현재 사용되지 않음	• DIVIDE 연산은 나눗셈과 비슷한 개념으로 왼쪽의 집합을 'XZ'로 나누었을 때, 즉 'XZ'를 모두 가지고 있는 'A'가 답이 되는 기능으로 현재 사용되지 않는다.

➤ 조인의 형태

조인의 형태	설명
INNER JOIN	• INNER JOIN은 OUTER(외부) JOIN과 대비하여 내부 JOIN이라고 하며 JOIN 조건에서 동일한 값이 있는 행만 반환
NATURAL JOIN	• NATURAL JOIN은 두 테이블 간의 동일한 이름을 갖는 모든 컬럼들에 대해 EQUI(=) JOIN을 수행
USING 조건절	• NATURAL JOIN에서는 모든 일치되는 컬럼들에 대해 JOIN이 이루어지지만, FROM 절의 USING 조건절을 이용하면 같은 이름을 가진 컬럼들 중에서 원하는 컬럼에 대해서만 선택적으로 EQUI JOIN을 할 수가 있음
ON 조건절	• JOIN 서술부(ON 조건절)와 비 JOIN 서술부(WHERE 조건 절)를 분리하여 이해가 쉬우며, 컬럼 명이 다르더라도 JOIN 조건을 사용할 수 있는 장점이 있음
CROSS JOIN	• CROSS JOIN은 E.F.CODD 박사가 언급한 일반 집합 연산자의 PRODUCT의 개념으로 테이블 간 JOIN 조건이 없는 경우 생길 수 있는 모든 데이터의 조합을 말함
OUTER JOIN	• INNER(내부) JOIN과 대비하여 OUTER(외부) JOIN이라고 불리며 JOIN 조건에서 동일한 값이 없는 행도 반환할 때 사용할 수 있음

➤ INNER JOIN 실습

```
SELECT A.EMP_NO
      , A.EMP_NM
      , A.ADDR
      , B.DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A, TB_DEPT B
WHERE A.DEPT_CD = B.DEPT_CD
      AND A.ADDR LIKE '%수원%'
ORDER BY A.EMP_NO
;
```

❖ 주소가 수원인 직원의 **사원번호, 사원명, 주소, 부서코드, 부서명**을 출력

EM P_N O	EM P_N M	ADD R	D EPT_CD	D EPT_N M
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	100002	지원팀
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	100003	기획팀
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	100004	디자인팀
1000000015	이정직	경기도 수원시 팔달구 인계동 124	100006	데이터팀
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	100007	개발팀
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	100009	운영팀
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	100012	인공지능팀
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	100013	빅데이터팀

➤ NATURAL JOIN 실습

```
SELECT A.EMP_NO
      , A.EMP_NM
      , A.ADDR
      , DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A NATURAL JOIN TB_DEPT B
WHERE A.ADDR LIKE '%수원%'
;
```

❖ 주소가 수원인 직원의 **사원번호, 사원명, 주소, 부서코드, 부서명**을 출력

EM P_N O	EM P_N M	ADD R	D EPT_CD	D EPT_N M
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	100002	지원팀
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	100003	기획팀
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	100004	디자인팀
1000000015	이정직	경기도 수원시 팔달구 인계동 124	100006	데이터팀
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	100007	개발팀
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	100009	운영팀
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	100012	인공지능팀
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	100013	빅데이터팀

❖ NATURAL조인은 두 테이블이 공통적으로 가지고 있는 DEPT_CD 컬럼으로 자동으로 조인된다.

➤ NATURAL JOIN 실습 - 조인 컬럼은 앨리어스를 가질 수 없음

```
SELECT A.EMP_NO  
      , A.EMP_NM  
      , A.ADDR  
      , B.DEPT_NM  
      , B.DEPT_CD  
FROM TB_EMP A NATURAL JOIN TB_DEPT B  
WHERE A.ADDR LIKE '%수원%'  
;
```

- ❖ NATURAL조인은 두 테이블이 공통적으로 가지고 있는 DEPT_CD 컬럼으로 자동으로 조인된다.
- ❖ 하지만 DEPT_CD컬럼을 SELECT절에 기재할 때 앨리어스를 주어서 에러가 났다.
- ❖ NATURAL 조인에서 조인 컬럼은 앨리어스를 주면 안된다.

SQL Error [25155] [99999]: ORA-25155: NATURAL 조인에 사용된 열은 식별자를 가질 수 없음

➤ USING절 실습

```
SELECT A.EMP_NO
      , A.EMP_NM
      , A.ADDR
      , B.DEPT_NM
      , DEPT_CD
FROM TB_EMP A JOIN TB_DEPT B USING (DEPT_CD)
WHERE A.ADDR LIKE '%수원%'
;
```

❖ 주소가 수원인 직원의 **사원번호, 사원명, 주소, 부서코드, 부서명**을 출력

- ❖ USING절에 두 테이블이 공통적으로 가지고 있는 DEPT_CD 컬럼을 기재한다.
- ❖ USING절에 들어가는 조인 컬럼에 앨리어스를 쓸 수 없다.

EM P_NO	EM P_NM	ADDR	DEPT_NM	DEPT_CD
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	지원팀	100002
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	기획팀	100003
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	디자인팀	100004
1000000015	이정직	경기도 수원시 팔달구 인계동 124	데이터팀	100006
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	개발팀	100007
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	운영팀	100009
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	인공지능팀	100012
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	빅데이터팀	100013

➤ USING절 실습 - 조인 컬럼에 앨리어스를 쓸수 없음

```
SELECT A.EMP_NO  
      , A.EMP_NM  
      , A.ADDR  
      , B.DEPT_NM  
      , DEPT_CD  
FROM TB_EMP A JOIN TB_DEPT B USING (B.DEPT_CD)  
WHERE A.ADDR LIKE '%수원%'  
;
```

- ❖ USING절에 두 테이블이 공통적으로 가지고 있는 DEPT_CD 컬럼을 기재한다.
- ❖ USING절에 들어가는 조인 컬럼에 앨리어스를 쓸 수 없다.

SQL Error [1748] [42000]: ORA-01748: 열명 그 자체만 사용할 수 있습니다

➤ ON절 실습

```
SELECT A.EMP_NO
      , A.EMP_NM
      , A.ADDR
      , B.DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A JOIN TB_DEPT B ON (A.DEPT_CD = B.DEPT_CD)
WHERE A.ADDR LIKE '%수원%'
;
```

❖ 주소가 수원인 직원의 **사원번호, 사원명, 주소, 부서 코드, 부서명**을 출력

- ❖ ON절에 **조인 컬럼인 DEPT_CD**컬럼을 기재한다.
- ❖ ON절 내에 **조인 컬럼에 앨리어스**를 사용해야 한다.
- ❖ **앨리어스를 정확히** 기재하지 않으면 에러가 발생한다.

EM P_NO	EM P_NM	ADD R	D EPT_CD	D EPT_NM
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	100002	지원팀
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	100003	기획팀
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	100004	디자인팀
1000000015	이정직	경기도 수원시 팔달구 인계동 124	100006	데이터팀
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	100007	개발팀
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	100009	운영팀
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	100012	인공지능팀
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	100013	빅데이터팀

➤ ON절 실습 - 앨리어스를 정확히 기재 하지 않은 경우

```
SELECT A.EMP_NO  
      , A.EMP_NM  
      , A.ADDR  
      , DEPT_CD  
      , B.DEPT_NM  
FROM TB_EMP A JOIN TB_DEPT B ON (A.DEPT_CD = B.DEPT_CD)  
WHERE A.ADDR LIKE '%수원%'  
;
```

- ❖ ON절에 **조인 컬럼인 DEPT_CD**컬럼을 기재한다.
- ❖ ON절 내에 **조인 컬럼에 앨리어스를 사용**해야 한다.
- ❖ **앨리어스를 정확히 기재하지 않으면 에러가 발생**한다.

SQL Error [918] [42000]: ORA-00918: 열의 정의가 애매합니다

➤ 3개의 테이블 조인

```
SELECT
    A.EMP_NO
  , A.EMP_NM
  , A.ADDR
  , B.DEPT_CD
  , B.DEPT_NM
  , C.CERTI_CD
FROM TB_EMP A
  , TB_DEPT B
  , TB_EMP_CERTI C
WHERE A.DEPT_CD = B.DEPT_CD
      AND A.ADDR LIKE '%수원%'
      AND A.EMP_NO = C.EMP_NO
ORDER BY A.EMP_NO;
```

❖ 3개의 테이블을 조인하는데 조인 조건은 2개가 들어갔다.

❖ 주소가 수원인 직원의 사원번호, 사원명, 주소, 부서코드, 부서명, 자격증코드를 출력

EMP_NO	EMP_NM	ADDR	DEPT_CD	DEPT_NM	CERT_LCD
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	100002	지원팀	100010
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	100002	지원팀	100009
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	100002	지원팀	100005
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	100003	기획팀	100009
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	100003	기획팀	100012
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	100003	기획팀	100005
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	100004	디자인팀	100015
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	100004	디자인팀	100004
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	100004	디자인팀	100015
1000000015	이정직	경기도 수원시 팔달구 인계동 124	100006	데이터팀	100004
1000000015	이정직	경기도 수원시 팔달구 인계동 124	100006	데이터팀	100018
1000000015	이정직	경기도 수원시 팔달구 인계동 124	100006	데이터팀	100011
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	100007	개발팀	100005
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	100007	개발팀	100008
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	100007	개발팀	100005
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	100009	운영팀	100014
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	100009	운영팀	100002
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	100009	운영팀	100013
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	100012	인공지능팀	100006
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	100012	인공지능팀	100009
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	100012	인공지능팀	100010
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	100013	빅데이터팀	100004
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	100013	빅데이터팀	100017
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	100013	빅데이터팀	100016

➤ 3개의 테이블 조인 - ANSI방식의 조인

```
SELECT A.EMP_NO
      , A.EMP_NM
      , A.ADDR
      , B.DEPT_NM
      , B.DEPT_CD
      , C.CERTI_CD
FROM TB_EMP A JOIN TB_DEPT B
ON (A.DEPT_CD = B.DEPT_CD)
JOIN TB_EMP_CERTI C
ON (A.EMP_NO = C.EMP_NO)
WHERE A.ADDR LIKE '%수원%'
;
```

❖ 3개의 테이블을 조인하는데 조인 조건은 2개가 들어갔다.

❖ 주소가 수원인 직원의 사원번호, 사원명, 주소, 부서코드, 부서명, 자격증코드를 출력

EMP_NO	EMP_NM	ADDR	DEPT_CD	DEPT_NM	CERT_LCD
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	지원팀	100002	100010
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	지원팀	100002	100009
1000000002	이현승	경기도 수원시 팔달구 매탄동 228	지원팀	100002	100005
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	기획팀	100003	100009
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	기획팀	100003	100012
1000000006	김혜진	경기도 수원시 팔달구 권선동 887	기획팀	100003	100005
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	디자인팀	100004	100015
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	디자인팀	100004	100004
1000000010	박혜령	경기도 수원시 팔달구 매탄동 987	디자인팀	100004	100015
1000000015	이정직	경기도 수원시 팔달구 인계동 124	데이터팀	100006	100004
1000000015	이정직	경기도 수원시 팔달구 인계동 124	데이터팀	100006	100018
1000000015	이정직	경기도 수원시 팔달구 인계동 124	데이터팀	100006	100011
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	개발팀	100007	100005
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	개발팀	100007	100008
1000000019	박정혜	경기도 수원시 팔달구 매탄동 987	개발팀	100007	100005
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	운영팀	100009	100014
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	운영팀	100009	100002
1000000024	박선영	경기도 수원시 팔달구 매탄2동 445	운영팀	100009	100013
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	인공지능팀	100012	100006
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	인공지능팀	100012	100009
1000000033	홍사기	경기도 수원시 팔달구 인계동 778	인공지능팀	100012	100010
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	빅데이터팀	100013	100004
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	빅데이터팀	100013	100017
1000000037	이현정	경기도 수원시 팔달구 매탄동 225	빅데이터팀	100013	100016

➤ **아우터 조인 - 실습 환경 구축**

```
INSERT INTO TB_DEPT VALUES ('100014', '4차산업혁명팀', '999999');
INSERT INTO TB_DEPT VALUES ('100015', '포스트코로나팀', '999999');

COMMIT;
```

- ❖ 부서 테이블에 부서 데이터를 추가함
- ❖ 새로 추가한 팀에는 어떠한 사원도 속하지 않은 상태임

```
ALTER TABLE SQLD.TB_EMP DROP CONSTRAINT FK_TB_EMP01;
```

- ❖ 참조 무결성 제약 조건(FK) 잠시 DROP

- ❖ 실습을 위해 사원 테이블에 신규 사원 5명을 추가함
- ❖ 추가되는 5명의 부서 코드는 존재하지 않는 부서인 "000000"로 인서트 시킴

```
INSERT INTO SQLD.TB_EMP T (T.EMP_NO, T.EMP_NM, T.BIRTH_DE, T.SEX_CD, T.ADDR, T.TEL_NO, T.DIRECT_MANAGER_EMP_NO,
T.FINAL_EDU_SE_CD, T.SAL_TRANS_BANK_CD, T.SAL_TRANS_ACCNT_NO, T.DEPT_CD, T.LUNAR_YN )
VALUES ('1000000041', '이순신', '19811201', '1', '경기도 용인시 수지구 죽전1동 435', '010-5456-7878', NULL, '006', '003',
'114-554-223433', '000000', 'N');
INSERT INTO SQLD.TB_EMP T (T.EMP_NO, T.EMP_NM, T.BIRTH_DE, T.SEX_CD, T.ADDR, T.TEL_NO, T.DIRECT_MANAGER_EMP_NO,
T.FINAL_EDU_SE_CD, T.SAL_TRANS_BANK_CD, T.SAL_TRANS_ACCNT_NO, T.DEPT_CD, T.LUNAR_YN )
VALUES ('1000000042', '정약용', '19820402', '1', '경기도 고양시 덕양구 화정동 231', '010-4054-6547', NULL, '004', '001',
'110-223-553453', '000000', 'Y');
INSERT INTO SQLD.TB_EMP T (T.EMP_NO, T.EMP_NM, T.BIRTH_DE, T.SEX_CD, T.ADDR, T.TEL_NO, T.DIRECT_MANAGER_EMP_NO,
T.FINAL_EDU_SE_CD, T.SAL_TRANS_BANK_CD, T.SAL_TRANS_ACCNT_NO, T.DEPT_CD, T.LUNAR_YN )
VALUES ('1000000043', '박지원', '19850611', '1', '경기도 수원시 팔달구 매탄동 553', '010-1254-1116', NULL, '004', '001',
'100-233-664234', '000000', 'N');
INSERT INTO SQLD.TB_EMP T (T.EMP_NO, T.EMP_NM, T.BIRTH_DE, T.SEX_CD, T.ADDR, T.TEL_NO, T.DIRECT_MANAGER_EMP_NO,
T.FINAL_EDU_SE_CD, T.SAL_TRANS_BANK_CD, T.SAL_TRANS_ACCNT_NO, T.DEPT_CD, T.LUNAR_YN )
VALUES ('1000000044', '장보고', '19870102', '1', '경기도 성남시 분당구 정자동 776', '010-1215-8784', NULL, '004', '002',
'180-345-556634', '000000', 'Y');
INSERT INTO SQLD.TB_EMP T (T.EMP_NO, T.EMP_NM, T.BIRTH_DE, T.SEX_CD, T.ADDR, T.TEL_NO, T.DIRECT_MANAGER_EMP_NO,
T.FINAL_EDU_SE_CD, T.SAL_TRANS_BANK_CD, T.SAL_TRANS_ACCNT_NO, T.DEPT_CD, T.LUNAR_YN )
VALUES ('1000000045', '김종서', '19880824', '1', '경기도 고양시 일산서구 백석동 474', '010-3687-1245', NULL, '004', '002',
'325-344-45345', '000000', 'Y');

COMMIT;
```

➤ **아우터 조인 - LEFT OUTER 조인**

```
SELECT A.EMP_NO
      , A.EMP_NM
      , B.DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A, TB_DEPT B
WHERE A.DEPT_CD IN ( '000000', '100001')
AND A.DEPT_CD = B.DEPT_CD(+)
;
```

❖ 현재 부서에 소속되어 있지 않은 직원들도 모두 집합에 포함됨
❖ 즉 TB_EMP(LEFT)는 다 나오고 TB_DEPT는 매칭되는 것만 나오게 됨

EM P_N O	EM P_N M	D EPT_CD	D EPT_N M
1000000001	이경오	100001	운영본부
1000000041	이순신	(N ULL)	(N ULL)
1000000042	정약용	(N ULL)	(N ULL)
1000000043	박지원	(N ULL)	(N ULL)
1000000044	장보고	(N ULL)	(N ULL)
1000000045	김종서	(N ULL)	(N ULL)

```
SELECT A.EMP_NO
      , A.EMP_NM
      , B.DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A
LEFT OUTER JOIN TB_DEPT B
ON (A.DEPT_CD = B.DEPT_CD)
WHERE A.DEPT_CD IN ( '000000', '100001')
;
```

❖ ANSI조인 방식의 **LEFT OUTER JOIN**임

EM P_N O	EM P_N M	D EPT_CD	D EPT_N M
1000000001	이경오	100001	운영본부
1000000041	이순신	(N ULL)	(N ULL)
1000000042	정약용	(N ULL)	(N ULL)
1000000043	박지원	(N ULL)	(N ULL)
1000000044	장보고	(N ULL)	(N ULL)
1000000045	김종서	(N ULL)	(N ULL)

➤ **아우터 조인 - RIGTH OUTER 조인**

```
SELECT A.EMP_NO
      , A.EMP_NM
      , B.DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A
      , TB_DEPT B
WHERE B.DEPT_CD IN ('100014', '100015', '100001')
      AND A.DEPT_CD(+) = B.DEPT_CD ;
```

```
SELECT A.EMP_NO
      , A.EMP_NM
      , B.DEPT_CD
      , B.DEPT_NM
FROM TB_EMP A
RIGHT OUTER JOIN TB_DEPT B
ON (A.DEPT_CD = B.DEPT_CD)
WHERE B.DEPT_CD IN ( '100014', '100015', '100001');
```

- ❖ 현재 아무런 사원을 가지고 있지 않은 부서도 모두 출력됨
- ❖ 즉 TB_DEPT(RIGHT)는 모두 나오고 TB_EMP는 매칭되는 집합만 출력됨

EM P_NO	EM P_NM	D EPT_CD	D EPT_NM
1000000001	이경오	100001	운영본부
(N ULL)	(N ULL)	100015	포스트코로나팀
(N ULL)	(N ULL)	100014	4차산업혁명팀

❖ ANSI조인 방식의 **RIGHT OUTER JOIN**임

EM P_NO	EM P_NM	D EPT_CD	D EPT_NM
1000000001	이경오	100001	운영본부
(N ULL)	(N ULL)	100015	포스트코로나팀
(N ULL)	(N ULL)	100014	4차산업혁명팀

➤ **아우터 조인 - FULL OUTER 조인**

```
SELECT
    A.EMP_NO
  , A.EMP_NM
  , B.DEPT_CD
  , B.DEPT_NM
FROM TB_EMP A
FULL OUTER JOIN TB_DEPT B ON (A.DEPT_CD = B.DEPT_CD)
WHERE 1 = 1
      AND ( A.EMP_NO IS NULL
            OR B.DEPT_CD IS NULL
            )
ORDER BY B.DEPT_CD DESC, A.EMP_NO DESC
;
```

❖ EMP_NO가 널이거나 DEPT_CD가 널인 것에 대한 조건을 주었다.
❖ 즉 EQUI 조인에 실패한 것들만을 추출하였음

EM P_NO	EM P_NM	DEPT_CD	DEPT_NM
1000000045	김종서	(NULL)	(NULL)
1000000044	장보고	(NULL)	(NULL)
1000000043	박지원	(NULL)	(NULL)
1000000042	정약용	(NULL)	(NULL)
1000000041	이순신	(NULL)	(NULL)
(NULL)	(NULL)	100015	포스트코로나팀
(NULL)	(NULL)	100014	4차산업혁명팀

➤ 아우터 조인 – 실습 종료 후 데이터 삭제 및 제약 조건 재설정

```
DELETE
  FROM TB_DEPT
 WHERE DEPT_CD IN ('100014', '100015');

DELETE FROM TB_EMP
WHERE EMP_NO IN ('1000000041', '1000000042', '1000000043', '1000000044', '1000000045');

COMMIT;

ALTER TABLE SQLD.TB_EMP ADD CONSTRAINT FK_TB_EMP01 FOREIGN KEY (DEPT_CD) REFERENCES SQLD.TB_DEPT (DEPT_CD);
```

Chapter 04. SQL 활용

4-2. 집합 연산자 (SET OPERATOR)

➤ 집합연산자의 종류

종류	설명
UNION	- 여러 개의 SQL문의 결과에 대한 합집합 - 중복된 행은 한개의 행으로 출력됨
UNION ALL	- 여러 개의 SQL문의 결과에 대한 합집합 - 중복된 행도 그대로 결과로 표시한다.
INTERSECT	여러 개의 SQL문의 대한 교집합 중복된 행은 하나로 표시한다.
EXCEPT	위의 SQL문의 집합에서 아래의 SQL문의 집합을 뺀 결과를 표시한다.

➤ UNION 실습

```
SELECT A.EMP_NO, A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19600101' AND '19691231'
UNION
SELECT A.EMP_NO, A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19700101' AND '197901231'
;
```

- ❖ 생년월일이 1960년대 및 1970년대에 태어난 직원의 **사원번호, 사원명, 생년월일**을 출력함
- ❖ 중복되는 행에 대해서는 **한 건 만 출력**

EM P_NO	EM P_NM	BIRTH_DE
1000000014	이관심	19780213
1000000023	이관심	19780213
1000000025	박호진	19720128
1000000026	김길정	19690524
1000000032	이준표	19771202
1000000034	김익정	19720128
1000000035	최창수	19690524
9999999999	김회장	19651105

- ❖ 총 8건의 레코드 출력
- ❖ 결과 집합이 정렬되어 있음

➤ UNION ALL 실습

```
SELECT A.EMP_NO, A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19600101' AND '19691231'
UNION ALL
SELECT A.EMP_NO, A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19700101' AND '197901231'
;
```

- ❖ 생년월일이 1960년대 및 1970년대에 태어난 직원의 **사원번호, 사원명, 생년월일**을 출력함
- ❖ 중복되는 행을 모두 출력(즉 **중복 된 것도 그대로 보여줌**)

EM P_NO	EM P_NM	BIRTH_DE
9999999999	김회장	19651105
1000000026	김길정	19690524
1000000035	최창수	19690524
1000000014	이관심	19780213
1000000023	이관심	19780213
1000000025	박호진	19720128
1000000032	이준표	19771202
1000000034	김익정	19720128

- ❖ 총 8건의 레코드 출력
- ❖ 결과 집합이 정렬되어 있지 않음

➤ UNION & UNION ALL 중복 행 실습

```
SELECT A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19600101' AND '19691231'
UNION ALL
SELECT A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19700101' AND '197901231'
;
```

- ❖ 생년월일이 1960년대 및 1970년대에 태어난 직원의 **사원명, 생년월일을 출력함**
- ❖ 동명이인으로 "**이관심**"이라는 직원이 중복된 것을 알 수 있음("이관심"이라는 2명의 직원은 이름과 생년월일까지 모두 같음)

EMP_NM	BIRTH_DE
김회장	19651105
김길정	19690524
최창수	19690524
이관심	19780213
이관심	19780213
박호진	19720128
이준표	19771202
김익정	19720128

- ❖ 총 8건의 레코드 출력
- ❖ 결과 집합이 정렬되어 있지 않음

```
SELECT A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19600101' AND '19691231'
UNION
SELECT A.EMP_NM, A.BIRTH_DE
FROM TB_EMP A
WHERE A.BIRTH_DE BETWEEN '19700101' AND '197901231'
;
```

- ❖ 생년월일이 1960년대 및 1970년대에 태어난 직원의 **사원명, 생년월일을 출력함**
- ❖ 동명이인인 "**이관심**" 직원은 중복된 행이 제거되는 과정에서 **한 건으로만** 보여짐

EMP_NM	BIRTH_DE
김길정	19690524
김익정	19720128
김회장	19651105
박호진	19720128
이관심	19780213
이준표	19771202
최창수	19690524

- ❖ 총 7건의 레코드 출력
- ❖ 결과 집합이 정렬되어 있음

➤ INTERSECT 실습

```
SELECT A.EMP_NO, A.EMP_NM, A.ADDR, B.CERTI_CD, C.CERTI_NM
FROM TB_EMP A , TB_EMP_CERTI B, TB_CERTI C
WHERE A.EMP_NO = B.EMP_NO
      AND B.CERTI_CD = C.CERTI_CD
      AND C.CERTI_NM = 'SQLD'
INTERSECT
SELECT A.EMP_NO, A.EMP_NM, A.ADDR, B.CERTI_CD, C.CERTI_NM
FROM TB_EMP A , TB_EMP_CERTI B, TB_CERTI C
WHERE A.EMP_NO = B.EMP_NO
      AND B.CERTI_CD = C.CERTI_CD
      AND A.ADDR LIKE '%용인%'
;
```

❖ SQLD자격증을 보유하면서 용인시에 사는 직원을 추출함

EMP_NO	EMP_NM	ADDR	CERT_CD	CERT_NM
100000013	이나라	경기도 용인시 수지구 풍덕천동 124	100001	SQLD

❖ INTERSECT 연산은 아래의 SQL문과 결과 집합이 동일함

```
SELECT A.EMP_NO, A.EMP_NM, A.ADDR, B.CERTI_CD, C.CERTI_NM
FROM TB_EMP A, TB_EMP_CERTI B, TB_CERTI C
WHERE A.EMP_NO = B.EMP_NO
      AND B.CERTI_CD = C.CERTI_CD
      AND C.CERTI_NM = 'SQLD'
      AND EXISTS ( SELECT 1
                    FROM TB_EMP K
                    WHERE K.EMP_NO = A.EMP_NO
                      AND K.ADDR LIKE '%용인%'
                  )
;
```

```
SELECT A.EMP_NO, A.EMP_NM, A.ADDR, B.CERTI_CD, C.CERTI_NM
FROM TB_EMP A, TB_EMP_CERTI B, TB_CERTI C
WHERE A.EMP_NO = B.EMP_NO
      AND B.CERTI_CD = C.CERTI_CD
      AND C.CERTI_NM = 'SQLD'
      AND A.ADDR LIKE '%용인%';
```

➤ MINUS 연산 실습

```
SELECT A.EMP_NO, A.EMP_NM, A.SEX_CD, A.DEPT_CD FROM TB_EMP A
MINUS
SELECT A.EMP_NO, A.EMP_NM, A.SEX_CD, A.DEPT_CD FROM TB_EMP A
WHERE A.DEPT_CD = '100001'
MINUS
SELECT A.EMP_NO, A.EMP_NM, A.SEX_CD, A.DEPT_CD FROM TB_EMP A
WHERE A.DEPT_CD = '100002'
MINUS
SELECT A.EMP_NO, A.EMP_NM, A.SEX_CD, A.DEPT_CD FROM TB_EMP A
WHERE A.DEPT_CD = '100003'
MINUS
SELECT A.EMP_NO, A.EMP_NM, A.SEX_CD, A.DEPT_CD FROM TB_EMP A
WHERE A.SEX_CD = '1'
```

- ❖ 전체 직원에서
- ❖ 부서 코드가 "100001" 인 직원들을 집합에서 제외
- ❖ 부서 코드가 "100002" 인 직원들을 집합에서 제외
- ❖ 부서 코드가 "100003" 인 직원들을 집합에서 제외
- ❖ 그 상태에서 성별이 남성인 직원들을 집합에서 제외

EMP_NO	EMP_NM	SEX_CD	DEPT_CD
0000000010	박혜령	2	000004
0000000011	최수자	2	000004
0000000013	이나라	2	000004
0000000016	이진실	2	000006
0000000018	박바른	2	000006
0000000019	박정혜	2	000007
0000000020	최정진	2	000007
0000000022	김순수	2	000007
0000000025	박호진	2	000009
0000000027	박이수	2	000009
0000000028	김나라	2	000010
0000000029	장나라	2	000010
0000000031	김사랑	2	000010
0000000034	김익정	2	000012
0000000036	박여진	2	000012
0000000037	이현정	2	000013
0000000038	김혜수	2	000013
0000000040	김여진	2	000013

Chapter 04. SQL 활용

4-3. 계층 형 질의와 SELF 조인

➤ 계층형 질의

- ① 테이블에 계층형 데이터가 존재하는 경우 데이터를 조회하기 위해서 계층형 질의(Hierarchical Query)를 사용
- ② 계층형 데이터란 동일 테이블에 계층적으로 상위와 하위 데이터가 포함된 데이터를 말한다.

직원

☐ # 직원번호

☐ * 직원명

☐ * 생년월일

☐ * 음력여부

☐ * 성별코드

☐ * 주소

☐ * 최종학력구분코드

☐ * 급여이체은행코드

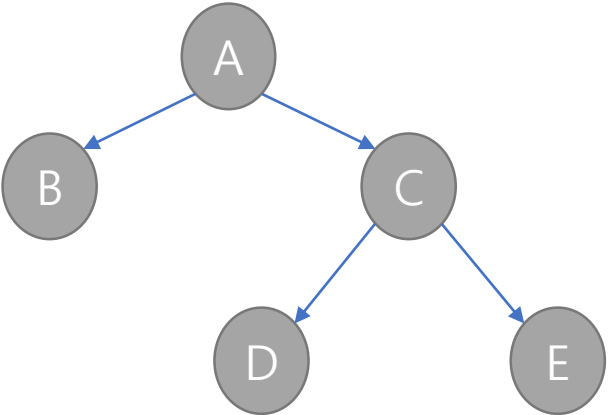
☐ * 급여이체계좌번호

☐ o 직속상사직원번호 (FK)

☐ * 부서코드

직속상사직원번호

순환 관계 모델



계층형 구조

직원	관리자
A	
B	A
C	A
D	C
E	C

샘플 데이터

➤ 오라클 계층 형 SQL

구분	설명
SELECT	조회하고자 하는 컬럼을 지정한다.
FROM TABLE	대상 테이블을 지정한다.
WHERE	모든 전개를 수행한 후에 지정된 조건을 만족하는 데이터만 추출한다.(필터링)
START WITH 조건	계층 구조 전개의 시작 위치를 지정하는 구문이다. 즉, 루트 데이터를 지정한다.
CONNECT BY [NOCYCLE] [PRIOR] A AND B	- CONNECT BY절은 다음에 전개될 자식 데이터를 지정하는 구문이다. - PRIOR (PK, 자식) = (FK, 부모) 형태를 사용하면 계층구조에서 부모 데이터에서 자식 데이터(부모 → 자식) 방향으로 전개하는 순방향 전개를 한다. - PRIOR (FK, 부모) = (PK, 자식) 형태를 사용하면 반대로 자식 데이터에서 부모 데이터(자식 →부모) 방향으로 전개하는 역방향 전개를 한다. - NOCYCLE를 추가하면 사이클이 발생한 이후의 데이터는 전개하지 않는다.
ORDER SIBLINGS BY 컬럼	형제 노드(동일 LEVEL) 사이에서 정렬을 수행한다.

➤ 계층 형 질의에서 사용되는 가상 컬럼

구분	설명
LEVEL	- 루트데이터면 1 - 그 하위 데이터 면 2 - 하위데이터가 있을 때마다 1씩 증가
CONNECT_BY_ISLEAF	전개과정에서 해당 데이터가 리프 데이터 이면 1 그렇지 않으면 0
CONNECT_BY_ROOT	현재행 기준으로 자신의 최고 상위 ROOT를 출력함

➤ 계층 형 SQL 실습

```
SELECT LEVEL LVL
      , LPAD(' ', 4*(LEVEL-1)) || EMP_NO || '(' || EMP_NM || ')' AS "조직인원"
      , A.DEPT_CD
      , B.DEPT_NM
      , CONNECT_BY_ISLEAF
FROM TB_EMP A, TB_DEPT B
WHERE A.DEPT_CD = B.DEPT_CD
START WITH A.DIRECT_MANAGER_EMP_NO IS NULL
CONNECT BY PRIOR A.EMP_NO = A.DIRECT_MANAGER_EMP_NO
;
```

- ❖ 관리자사원번호가 널인 값을 첫 시작 값으로 하였음
- ❖ PRIOR [자식, PK] = [부모, FK]로 순방향(부모->자식) 전개를 하였음
- ❖ CONNECT_BY_ISLEAF를 이용해서 LEAF노드인 경우 1을 출력함

LVL	조직인원	DEPT_CD	DEPT_NM	CONNECT_BY_ISLEAF
1	9999999999(김 회장)	999999	회장실	0
2	1000000001(이경오)	100001	운영본부	0
3	1000000002(이현승)	100002	지원팀	0
4	1000000003(이정수)	100002	지원팀	1
4	1000000004(이승준)	100002	지원팀	1
4	1000000005(김현희)	100002	지원팀	1
중간생략 ...				
3	1000000037(이현정)	100013	빅데이터팀	0
4	1000000038(김혜수)	100013	빅데이터팀	1
4	1000000039(이박력)	100013	빅데이터팀	1
4	1000000040(김여진)	100013	빅데이터팀	1

➤ 계층 형 SQL 실습 – START WITH의 변경

```
SELECT LEVEL LVL
      , LPAD(' ', 4*(LEVEL-1)) || EMP_NO || '(' || EMP_NM || ')' AS "조직인원"
      , A.DEPT_CD
      , B.DEPT_NM
      , CONNECT_BY_ISLEAF
FROM TB_EMP A, TB_DEPT B
WHERE A.DEPT_CD = B.DEPT_CD
START WITH A.EMP_NM = '이경오'
CONNECT BY PRIOR A.EMP_NO = A.DIRECT_MANAGER_EMP_NO
;
```

- ❖ 사원명이 이경오인 행을 처음으로 시작함
- ❖ PRIOR [자식, PK] = [부모, FK] 로 순방향(부모->자식) 전개를 하였음
- ❖ CONNECT_BY_ISLEAF를 이용해서 LEAF노드인 경우 1을 출력함

LVL	조직인원	DEPT_CD	DEPT_NM	CONNECT_BY_ISLEAF
1	1000000001(이경오)	100001	운영본부	0
2	1000000002(이현승)	100002	지원팀	0
3	1000000003(이정수)	100002	지원팀	1
3	1000000004(이승준)	100002	지원팀	1
3	1000000005(김현희)	100002	지원팀	1
2	1000000006(김혜진)	100003	기획팀	0
3	1000000007(이순자)	100003	기획팀	1
3	1000000008(김려원)	100003	기획팀	1
3	1000000009(박태범)	100003	기획팀	1
2	1000000010(박혜령)	100004	디자인팀	0
3	1000000011(최수자)	100004	디자인팀	1
3	1000000012(김호형)	100004	디자인팀	1
3	1000000013(이나라)	100004	디자인팀	1

➤ 계층형 SQL 실습 – CONNECT_BY_ROOT의 사용

```
SELECT LEVEL LVL
      , LPAD(' ', 4*(LEVEL-1)) || EMP_NO || '(' || EMP_NM || ')' AS "조직인원"
      , A.DEPT_CD
      , B.DEPT_NM
      , CONNECT_BY_ISLEAF
      , CONNECT_BY_ROOT A.EMP_NO AS "최상위관리자"
FROM TB_EMP A, TB_DEPT B
WHERE A.DEPT_CD = B.DEPT_CD
START WITH A.DIRECT_MANAGER_EMP_NO IS NULL
CONNECT BY PRIOR A.EMP_NO = A.DIRECT_MANAGER_EMP_NO;
```

- ❖ 최고관리자를 START WITH로 시작함
- ❖ PRIOR [자식, PK] = [부모, FK]로 순방향(부모->자식) 전개를 하였음
- ❖ CONNECT_BY_ISLEAF를 이용해서 LEAF노드인 경우 1을 출력함
- ❖ CONNECT_BY_ROOT를 이용하여 최상위 관리자를 출력함

LVL	조직인원	DEPT_CD	DEPT_NM	CONNECT_BY_ISLEAF	최상위관리자
1	9999999999(김 회장)	999999	회장실	0	9999999999
2	1000000001(이경오)	100001	운영본부	0	9999999999
3	1000000002(이현승)	100002	지원팀	0	9999999999
4	1000000003(이정수)	100002	지원팀	1	9999999999
4	1000000004(이승준)	100002	지원팀	1	9999999999
4	1000000005(김현희)	100002	지원팀	1	9999999999
중간생략 ...					
2	1000000032(이준표)	100011	신사업본부	0	9999999999
3	1000000033(홍사기)	100012	인공지능팀	0	9999999999
4	1000000034(김익정)	100012	인공지능팀	1	9999999999
4	1000000035(최창수)	100012	인공지능팀	1	9999999999
4	1000000036(박여진)	100012	인공지능팀	1	9999999999
3	1000000037(이현정)	100013	빅데이터팀	0	9999999999
4	1000000038(김혜수)	100013	빅데이터팀	1	9999999999
4	1000000039(이박력)	100013	빅데이터팀	1	9999999999
4	1000000040(김여진)	100013	빅데이터팀	1	9999999999

➤ 계층형 SQL 실습 – SYS_CONNECT_BY_PATH의 사용

```
SELECT LEVEL LVL
      , LPAD(' ', 4*(LEVEL-1)) || EMP_NO || '(' || EMP_NM || ')' AS "조직인원"
      , A.DEPT_CD
      , B.DEPT_NM
      , CONNECT_BY_ISLEAF
      , CONNECT_BY_ROOT A.EMP_NO AS "최상위관리자"
      , SYS_CONNECT_BY_PATH(EMP_NO || '(' || EMP_NM || ')', '/') AS "조직인원경로"
FROM TB_EMP A, TB_DEPT B
WHERE A.DEPT_CD = B.DEPT_CD
START WITH A.EMP_NM = '이경오'
CONNECT BY PRIOR A.EMP_NO = A.DIRECT_MANAGER_EMP_NO;
```

- ❖ 사원명이 이경오인 행을 START WITH로 시작함
- ❖ PRIOR [자식, PK] = [부모, FK]로 순방향(부모->자식) 전개를 하였음
- ❖ CONNECT_BY_ISLEAF를 이용해서 LEAF노드인 경우 1을 출력함
- ❖ CONNECT_BY_ROOT를 이용하여 최상위 관리자를 출력함
- ❖ SYS_CONNECT_BY_PATH 함수를 이용하여 조직인원경로를 출력함

LVL	조직인원	DEPT_CD	DEPT_NM	CONNECT_BY_ISLEAF	최상위관리자	조직인원경로
1	1000000001(이경오)	100001	운영본부	0	1000000001	/1000000001(이경오)
2	1000000002(이현승)	100002	지원팀	0	1000000001	/1000000001(이경오)/1000000002(이현승)
3	1000000003(이정수)	100002	지원팀	1	1000000001	/1000000001(이경오)/1000000002(이현승)/1000000003(이정수)
3	1000000004(이승준)	100002	지원팀	1	1000000001	/1000000001(이경오)/1000000002(이현승)/1000000004(이승준)
3	1000000005(김현희)	100002	지원팀	1	1000000001	/1000000001(이경오)/1000000002(이현승)/1000000005(김현희)
2	1000000006(김혜진)	100003	기획팀	0	1000000001	/1000000001(이경오)/1000000006(김혜진)
3	1000000007(이순자)	100003	기획팀	1	1000000001	/1000000001(이경오)/1000000006(김혜진)/1000000007(이순자)
3	1000000008(김려원)	100003	기획팀	1	1000000001	/1000000001(이경오)/1000000006(김혜진)/1000000008(김려원)
3	1000000009(박태범)	100003	기획팀	1	1000000001	/1000000001(이경오)/1000000006(김혜진)/1000000009(박태범)
2	1000000010(박혜령)	100004	디자인팀	0	1000000001	/1000000001(이경오)/1000000010(박혜령)
3	1000000011(최수자)	100004	디자인팀	1	1000000001	/1000000001(이경오)/1000000010(박혜령)/1000000011(최수자)
3	1000000012(김호형)	100004	디자인팀	1	1000000001	/1000000001(이경오)/1000000010(박혜령)/1000000012(김호형)
3	1000000013(이나라)	100004	디자인팀	1	1000000001	/1000000001(이경오)/1000000010(박혜령)/1000000013(이나라)

➤ 계층형 SQL 실습 – SELF 조인의 활용

```
SELECT A.EMP_NO "사원번호"
      , A.EMP_NM "사원명"
      , A.DIRECT_MANAGER_EMP_NO "관리자사원번호"
      , (SELECT L.EMP_NM FROM TB_EMP L WHERE L.EMP_NO = A.DIRECT_MANAGER_EMP_NO) AS "관리자사원명"
      , B.DIRECT_MANAGER_EMP_NO AS "차상위관리자사원번호"
      , (SELECT L.EMP_NM FROM TB_EMP L WHERE L.EMP_NO = B.DIRECT_MANAGER_EMP_NO) AS "차상위관리자사원명"
FROM TB_EMP A INNER JOIN TB_EMP B
ON (A.DIRECT_MANAGER_EMP_NO = B.EMP_NO)
JOIN TB_DEPT C
ON (A.DEPT_CD = C.DEPT_CD)
;
```

사원번호	사원명	관리자사원번호	관리자사원명	차상위관리자사원번호	차상위관리자사원명
1000000006	김혜진	1000000001	이경오	9999999999	김회장
1000000010	박혜령	1000000001	이경오	9999999999	김회장
1000000002	이현승	1000000001	이경오	9999999999	김회장
1000000004	이승준	1000000002	이현승	1000000001	이경오
1000000003	이정수	1000000002	이현승	1000000001	이경오
1000000005	김현희	1000000002	이현승	1000000001	이경오
1000000007	이순자	1000000006	김혜진	1000000001	이경오
1000000008	김려원	1000000006	김혜진	1000000001	이경오
1000000009	박태범	1000000006	김혜진	1000000001	이경오
1000000012	김호형	1000000010	박혜령	1000000001	이경오
1000000013	이나라	1000000010	박혜령	1000000001	이경오
1000000011	최수자	1000000010	박혜령	1000000001	이경오
중간생략 ...					
1000000001	이경오	9999999999	김회장		
1000000014	이관심	9999999999	김회장		
1000000023	이관심	9999999999	김회장		
1000000032	이준표	9999999999	김회장		

- ❖ SELF 조인은 동일한 테이블끼리의 조인을 의미함
- ❖ SELF 조인을 이용해서 계층형의 데이터를 출력할 수 있음
- ❖ SELF 조인 시 INNER 조인을 하여 관리자가 존재하지 않는 김회장의 레코드는 결과 집합에 포함되지 않음

➤ 계층형 SQL 실습 – SELF 조인 및 OUTER 조인의 활용

```
SELECT A.EMP_NO "직원번호"
      , A.EMP_NM "직원명"
      , A.DIRECT_MANAGER_EMP_NO "관리자직원번호"
      , (SELECT L.EMP_NM FROM TB_EMP L WHERE L.EMP_NO = A.DIRECT_MANAGER_EMP_NO) AS "관리자직원명"
      , B.DIRECT_MANAGER_EMP_NO AS "차상위관리자직원번호"
      , (SELECT L.EMP_NM FROM TB_EMP L WHERE L.EMP_NO = B.DIRECT_MANAGER_EMP_NO) AS "차상위관리자직원명"
FROM TB_EMP A LEFT OUTER JOIN TB_EMP B
ON (A.DIRECT_MANAGER_EMP_NO = B.EMP_NO)
JOIN TB_DEPT C
ON (A.DEPT_CD = C.DEPT_CD)
;
```

❖ SELF 조인을 하면서 OUTER 조인까지 하여 상위 관리자가 없어서 결과 집합에 나오지 않았던 김회장의 행까지 출력 하게 됨

직원번호	직원명	관리자직원번호	관리자직원명	차상위관리자직원번호	차상위관리자직원명
1000000001	이경오	9999999999	김 회장		
1000000014	이관심	9999999999	김 회장		
1000000023	이관심	9999999999	김 회장		
1000000032	이준표	9999999999	김 회장		
1000000002	이현승	1000000001	이경오	9999999999	김 회장
1000000006	김혜진	1000000001	이경오	9999999999	김 회장
1000000010	박혜령	1000000001	이경오	9999999999	김 회장
중간생략 ...					
9999999999	김 회장				

Chapter 04. SQL 활용

4-4. 서브 쿼리

➤ 서브 쿼리란

- ① 서브 쿼리(Subquery)란 하나의 SQL문안에 포함되어 있는 또 다른 SQL문을 말한다.
- ② 조인은 조인에 참여하는 모든 테이블이 대등한 관계에 있기 때문에 조인에 참여하는 모든 테이블의 컬럼을 어느 위치에서라도 자유롭게 사용할 수 있다. 그러나 서브 쿼리는 메인 쿼리의 컬럼을 모두 사용할 수 있지만 메인 쿼리는 서브 쿼리의 컬럼을 사용할 수 없다.

➤ 서브 쿼리 사용시 주의점

- ① 서브쿼리를 괄호로 감싸서 사용한다.
- ② 서브 쿼리는 단일 행(Single Row) 또는 복수 행(Multiple Row) 비교 연산자와 함께 사용 가능하다.
- ③ 단일 행 비교 연산자는 서브 쿼리의 결과가 반드시 1건 이하이어야 하고 복수 행 비교 연산자는 서브 쿼리의 결과 건수와 상관 없다.
- ④ 서브쿼리에서는 ORDER BY를 사용하지 못한다. ORDER BY절은 SELECT절에서 오직 한 개만 올 수 있기 때문에 ORDER BY절은 메인 쿼리의 마지막 문장에 위치해야 한다.

➤ 서브 쿼리가 사용 가능한 위치

- ① SELECT 절 - FROM 절 - WHERE 절 - HAVING 절 - ORDER BY 절
- ② INSERT문의 VALUES 절 - UPDATE문의 SET 절

➤ 동작 방식에 따른 서브 쿼리 분류

동작 방식	설명
비 연관 서브 쿼리	- 서브 쿼리가 메인 쿼리의 컬럼을 가지고 있지 않은 형태의 서브 쿼리임 - 메인 쿼리에 값을 제공하기 위한 목적으로 주로 사용
연관 서브 쿼리	- 서브 쿼리가 메인 쿼리의 값을 가지고 있는 형태의 서브쿼리임 - 일반적으로 메인 쿼리가 먼저 수행되어 읽혀진 데이터를 서브쿼리에서 조건이 맞는지 확인하고자 할 때 주로 사용

➤ 반환 형태에 따른 서브 쿼리 분류

반환 형태	설명
단일 행 서브 쿼리	- 서브 쿼리의 실행 결과가 항상 1건 이하인 서브쿼리를 의미한다. - 항상 비교 연산자와 함께 사용된다. (=, <, <=, >, >=, <>)
다중 행 서브 쿼리	- 서브 쿼리의 실행 결과가 여러 건인 서브쿼리를 의미한다. - 다중 행 서브 쿼리는 다중 행 비교 연산자와 함께 사용된다. (IN, ALL, ANY, SOME, EXISTS)
다중 컬럼 서브 쿼리	- 서브 쿼리의 실행 결과로 여러 컬럼을 반환한다. - 메인 쿼리의 조건 절에 여러 컬럼을 동시에 비교 할 수 있다. - 서브 쿼리와 메인 쿼리의 컬럼 수와 컬럼 순서가 동일해야 한다

➤ 단일 행 서브 쿼리 실습

```
SELECT A.EMP_NO, A.EMP_NM, A.DEPT_CD
FROM TB_EMP A
WHERE A.DEPT_CD =
(
    SELECT DEPT_CD
    FROM TB_EMP
    WHERE EMP_NO = '1000000005'
);
```

```
SELECT A.EMP_NO, B.EMP_NM, A.PAY_DE, A.PAY_AMT
FROM SQLD.TB_SAL_HIS A, TB_EMP B
WHERE A.PAY_DE = '20200525'
AND A.PAY_AMT >=
(
    SELECT AVG(K.PAY_AMT)
    FROM SQLD.TB_SAL_HIS K
    WHERE K.PAY_DE = '20200525'
)
AND A.EMP_NO = B.EMP_NO;
```

❖ 2020년5월 기준 평균 이상의급여를 받고 있는 직원들의
리스트

❖ EMP_NO가 "1000000005"가 속한 팀의 팀원을 조회하는 SQL문

EM P_NO	EM P_NM	D EPT_CD
1000000002	이현승	100002
1000000003	이정수	100002
1000000004	이승준	100002
1000000005	김현희	100002

EM P_NO	EM P_NM	PAY_DE	PAY_AM T
1000000003	이정수	20200525	4440000
1000000004	이승준	20200525	4910000
1000000005	김현희	20200525	4320000
1000000007	이순자	20200525	5000000
1000000009	박태범	20200525	4100000
1000000013	이나라	20200525	4340000
1000000015	이정직	20200525	5100000
1000000016	이진실	20200525	4340000
중간생략			
1000000031	김사랑	20200525	5150000
1000000032	이준표	20200525	5570000
1000000034	김익정	20200525	5890000
1000000035	최창수	20200525	4350000
1000000037	이현정	20200525	5140000
1000000040	김여진	20200525	4210000
9999999999	김 회장	20200525	5470000

➤ 다중 행 서브 쿼리 실습

```
SELECT A.EMP_NO, COUNT(*) CNT
FROM TB_EMP_CERTI A
WHERE A.CERTI_CD
IN
(
SELECT K.CERTI_CD
FROM SQLD.TB_CERTI K
WHERE K.ISSUE_INSTI_NM = '한국데이터베이스진흥원'
)
GROUP BY A.EMP_NO
ORDER BY A.EMP_NO
;
```

❖ 한국데이터베이스진흥원에서 발급한 자격증을 가지고 있는 **사원 번호** 및 **보유 자격 개수**를 출력하는 SQL문

EM P_NO	CNT
1000000001	1
1000000002	1
1000000004	1
1000000005	2
1000000006	1
1000000007	1
1000000008	1
1000000009	2
중간생략 ...	
1000000034	2
1000000036	1
1000000037	1
1000000038	1
1000000040	1
9999999999	1

```
SELECT A.EMP_NO, COUNT(*) CNT
FROM TB_EMP_CERTI A
WHERE A.CERTI_CD
=
(
SELECT K.CERTI_CD
FROM SQLD.TB_CERTI K
WHERE K.ISSUE_INSTI_NM = '한국데이터베이스진흥원'
)
GROUP BY A.EMP_NO
ORDER BY A.EMP_NO
;
```

❖ 다중 행 서브 쿼리에 **= 연산자**를 써서 SQL에러가 발생함

SQL Error [1427] [21000]: ORA-01427: 단일 행 하위 질의에 2개 이상의 행이 리턴 되었습니다.

➤ 다중컬럼 서브쿼리 실습

```
SELECT A.EMP_NO
      , A.EMP_NM
      , A.DEPT_CD
      , B.DEPT_NM
      , A.BIRTH_DE
FROM TB_EMP A
     , TB_DEPT B
WHERE (A.DEPT_CD, A.BIRTH_DE) IN
(
  SELECT K.DEPT_CD, MIN(K.BIRTH_DE) AS MIN_BIRTH_DE
  FROM TB_EMP K
  GROUP BY K.DEPT_CD
  HAVING COUNT(*) > 1
)
AND A.DEPT_CD = B.DEPT_CD
ORDER BY A.EMP_NO
;
```

❖ 한 부서에 2명 이상 있는 부서 중에서 각 부서의 생일 기준 최고참을 출력하는 SQL문

EM P_NO	EM P_NM	DEPT_CD	DEPT_NM	BIRTH_DE
1000000004	이승준	100002	지원팀	19870623
1000000006	김혜진	100003	기획팀	19840715
1000000013	이나라	100004	디자인팀	19871201
1000000018	박바른	100006	데이터팀	19820629
1000000022	김순수	100007	개발팀	19871201
1000000026	김길정	100009	운영팀	19690524
1000000031	김사랑	100010	개발팀	19811201
1000000035	최창수	100012	인공지능팀	19690524
1000000040	김여진	100013	빅데이터팀	19871205

➤ EXISTS문 서브 쿼리 실습

```
SELECT A.DEPT_CD, A.DEPT_NM
FROM TB_DEPT A
WHERE EXISTS ( SELECT 1
                FROM TB_EMP K
                WHERE K.DEPT_CD = A.DEPT_CD
                  AND K.ADDR LIKE '%강남%'
              )
;
```

❖ 직원들 중 주소가 강남인 직원이 소속된 부서 코드와 부서명을 출력

DEPT_CD	DEPT_NM
100009	운영팀
100010	개발팀

➤ 스칼라 서브 쿼리 실습

```
SELECT A.EMP_NO
      , (SELECT L.EMP_NM FROM TB_EMP L WHERE L.EMP_NO = A.EMP_NO) AS EMP_NM
      , A.CERTI_CD
      , (SELECT L.CERTI_NM FROM TB_CERTI L WHERE L.CERTI_CD = A.CERTI_CD) AS CERTI_NM
FROM TB_EMP_CERTI A
WHERE A.CERTI_CD IN
      (
        SELECT K.CERTI_CD
        FROM SQLD.TB_CERTI K
        WHERE K.ISSUE_INSTI_NM = '한국데이터베이스진흥원'
      )
ORDER BY CERTI_NM
;
```

❖ 한국데이터베이스진흥원에서 발급한 자격증을 가지고 있는 사원의 사원 번호, 사원명, 자격증 코드, 자격증명을 출력

EMP_NO	EMP_NM	CERT_CD	CERT_NM
1000000017	이경손	100006	ADP
1000000033	홍사기	100006	ADP
1000000022	김순수	100006	ADP
1000000022	김순수	100006	ADP
1000000018	박바른	100006	ADP
1000000020	최정진	100006	ADP
1000000016	이진실	100006	ADP
중간생략 ...			
1000000027	박이수	100006	ADP
1000000013	이나라	100001	SQLD
1000000025	박호진	100001	SQLD
1000000005	김현희	100002	SQLP
1000000001	이경오	100002	SQLP
1000000024	박선영	100002	SQLP
1000000034	김익정	100002	SQLP
1000000008	김려원	100002	SQLP

➤ 인라인 뷰 서브 쿼리 실습

```
SELECT B.EMP_NO
      , (SELECT L.EMP_NM FROM TB_EMP L WHERE L.EMP_NO = B.EMP_NO) AS EMP_NM
      , B.CERTI_CD
      , (SELECT L.CERTI_NM FROM TB_CERTI L WHERE L.CERTI_CD = B.CERTI_CD) AS CERTI_NM
FROM
  (
    SELECT K.CERTI_CD
      FROM SQLD.TB_CERTI K
     WHERE K.ISSUE_INSTI_NM = '한국데이터베이스진흥원'
  ) A
  , TB_EMP_CERTI B
WHERE A.CERTI_CD = B.CERTI_CD
ORDER BY CERTI_NM
;
```

❖ 한국데이터베이스진흥원에서 발급한 자격증을 가지고 있는 사원의 사원 번호, 사원명, 자격증 코드, 자격증명을 출력

EM P_N O	EM P_N M	CERT LCD	CERT LN M
1000000017	이경손	100006	ADP
1000000033	홍사기	100006	ADP
1000000022	김순수	100006	ADP
1000000022	김순수	100006	ADP
1000000018	박바른	100006	ADP
1000000020	최정진	100006	ADP
1000000016	이진실	100006	ADP
1000000027	박이수	100006	ADP
1000000006	김혜진	100005	AD SP
1000000040	김여진	100005	AD SP
1000000032	이준표	100005	AD SP
1000000032	이준표	100005	AD SP
1000000004	이승준	100005	AD SP
1000000028	김나라	100005	AD SP
1000000019	박정혜	100005	AD SP
중간생략			
1000000007	이순자	100003	D A SP
1000000031	김사랑	100003	D A SP
1000000013	이나라	100001	SQ LD
1000000025	박호진	100001	SQ LD
1000000005	김현희	100002	SQ LP
1000000001	이경오	100002	SQ LP
1000000024	박선영	100002	SQ LP
1000000034	김익정	100002	SQ LP
1000000008	김려원	100002	SQ LP

➤ HAVING절에서 서브 쿼리

```
SELECT B.DEPT_CD
      , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = B.DEPT_CD) "부서명"
      , AVG(A.PAY_AMT) "평균급여"
FROM SQLD.TB_SAL_HIS A, TB_EMP B
WHERE A.PAY_DE = '20200525'
      AND A.EMP_NO = B.EMP_NO
GROUP BY B.DEPT_CD
HAVING AVG(A.PAY_AMT) >
(
  SELECT AVG(K.PAY_AMT)
    FROM SQLD.TB_SAL_HIS K, TB_EMP J
   WHERE K.PAY_DE = '20200525'
        AND K.EMP_NO = J.EMP_NO
        AND J.DEPT_CD = '100004'
)
ORDER BY "평균급여" DESC
;
```

D EPT_CD	부서명	평균급여
100011	신사업본부	5570000
999999	회장실	5470000
100010	개발팀	4890000
100002	지원팀	4357500
100009	운영팀	4287500
100012	인공지능팀	4137500
100006	데이터팀	4105000
100013	빅데이터팀	3982500
100003	기획팀	3802500
100008	솔루션사업본부	3710000
100001	운영본부	3700000

❖ 2020년 05월의 "100004"(디자인팀) 부서의 평균 급여보다 평균급여가 높은 부서의 부서 코드와 부서명을 구하는 SQL문

➤ UPDATE문에 사용되는 서브 쿼리

```
ALTER TABLE TB_EMP ADD(DEPT_NM VARCHAR2(150));

UPDATE TB_EMP A
  SET A.DEPT_NM = ( SELECT K.DEPT_NM FROM TB_DEPT K WHERE K.DEPT_CD = A.DEPT_CD)
;

COMMIT;
```

```
SELECT EMP_NO, EMP_NM, DEPT_CD, DEPT_NM
FROM TB_EMP
WHERE ROWNUM <= 10;
```

EM P_NO	EM P_NM	DEPT_CD	DEPT_NM
9999999999	김 회 장	999999	회 장 실
1000000001	이 경 오	100001	운 영 본 부
1000000002	이 현 승	100002	지 원 팀
1000000003	이 정 수	100002	지 원 팀
1000000004	이 승 준	100002	지 원 팀
1000000005	김 현 희	100002	지 원 팀
1000000006	김 혜 진	100003	기 획 팀
1000000007	이 순 자	100003	기 획 팀
1000000008	김 려 원	100003	기 획 팀
1000000009	박 태 범	100003	기 획 팀

```
ALTER TABLE TB_EMP DROP COLUMN DEPT_NM;
```

❖ TB_EMP테이블에 DEPT_NM 컬럼을 추가하고 UPDATE문을 이용하여 DEPT_NM 의 값을 일괄적으로 UPDATE 함

➤ INSERT문에 사용되는 서브쿼리

```
CREATE TABLE INSERT_SUBQUERY_TEST
(
    EMP_NO CHAR(10)
    , MAX_PAY_AMT NUMBER(15)
);

INSERT INTO INSERT_SUBQUERY_TEST
VALUES ('1000000001', (SELECT MAX(PAY_AMT) FROM TB_SAL_HIS WHERE EMP_NO = '1000000001'))
;

COMMIT;
```

```
SELECT * FROM INSERT_SUBQUERY_TEST;
```

EMP_NO	MAX_PAY_AMT
1000000001	5890000

```
DROP TABLE INSERT_SUBQUERY_TEST;
```

- ❖ INSERT_SUBQUERY_TEST테이블을 생성하고
- ❖ 사원 번호가 "1000000001"인 직원의 최고 급여를 삽입함

뷰 사용의 장점

장점	설명
독립성	• 테이블 구조가 변경되어도 뷰를 사용하는 응용프로그램은 변경하지 않아도 된다.
편리성	• 복잡한 질의를 뷰로 생성함으로써 관련 질의를 단순하게 작성할 수 있다. 또한 해당 형태의 SQL문을 자주 사용할 때 뷰를 이용하면 편리하게 사용할 수 있다.
보안성	• 직원의 급여 정보와 같이 숨기고 싶은 정보가 존재한다면, 뷰를 생성 할 때 해당 컬럼을 빼고 생성함으로써 사용자에게 정보를 감출 수 있다.

뷰 사용 실습

```
CREATE VIEW V_TB_SAL_HIS_MAX_BY_EMP_NO
AS
SELECT A.EMP_NO, A.EMP_NM, B.DEPT_CD, B.DEPT_NM
, MAX(C.PAY_AMT) AS MAX_PAY_AMT
FROM TB_EMP A , TB_DEPT B, TB_SAL_HIS C
WHERE A.DEPT_CD = B.DEPT_CD
AND A.EMP_NO = C.EMP_NO
GROUP BY A.EMP_NO, A.EMP_NM, B.DEPT_CD, B.DEPT_NM
;
```

```
SELECT * FROM V_TB_SAL_HIS_MAX_BY_EMP_NO;
```

```
DROP VIEW V_TB_SAL_HIS_MAX_BY_EMP_NO;
```

EMP_NO	EMP_NM	DEPT_CD	DEPT_NM	PAY_AMT
1000000003	이정수	100002	지원팀	5560000
1000000005	김현희	100002	지원팀	5860000
1000000009	박태범	100003	기획팀	5840000
1000000015	이정직	100006	데이터팀	5960000
1000000022	김순수	100007	개발팀	5690000
1000000030	이규호	100010	개발팀	5710000
1000000037	이현정	100013	빅데이터팀	5870000
1000000004	이승준	100002	지원팀	5800000
1000000019	박정혜	100007	개발팀	5980000
중간생략...				
1000000033	홍사기	100012	인공지능팀	5830000
1000000001	이경오	100001	운영본부	5890000
1000000002	이현승	100002	지원팀	5560000
1000000029	장나라	100010	개발팀	5980000
1000000034	김익정	100012	인공지능팀	5890000
1000000038	김혜수	100013	빅데이터팀	5850000
1000000039	이박력	100013	빅데이터팀	5940000

Chapter 04. SQL 활용

4-5. 그룹 함수 (GROUP FUNCTION)

➤ 그룹 함수란?

- ① 그룹 함수를 이용하여 특정 집합의 소계, 중계, 합계, 총 합계를 구할 수 있다.
- ② 즉 이러한 합계를 계산하기 위해서 기존에 들어갔던 다양한 노력들이 그룹 함수를 이용하여 간단하게 처리할 수 있게 되었다.

➤ 그룹 함수의 종류

종류	설명
ROLLUP	• 소 그룹간의 소계를 계산하는 기능 • ROLLUP함수 내에 인자로 지정된 GRUPING 컬럼은 SUBTOTAL을 생성하는데 사용된다. • GROUPING 컬럼의 수가 N이라고 했을 때 N+1의 SUBTOTAL이 생성된다. • ROLLUP함수 내의 인자의 순서가 바뀌면 결과도 바뀌게 된다.(ROLLUP은 계층 구조임)
CUBE	• 다차원적인 소계를 계산하는 기능 • 결합 가능한 모든 값에 대하여 다차원 집계를 생성 • CUBE함수 내에 컬럼이 N개라면 2의 N승만큼의 SUBTOTAL이 생성됨 • 시스템에 많은 부담을 주기때문에 사용상 주의가 필요함
GROUPING SETS	• 특정 항목에 대한 소계를 계산하는 기능

➤ 그룹 함수 실습 - 합계 데이터 출력

```
SELECT A.DEPT_CD "부서코드"
      , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.DEPT_CD) AS "부서명"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
      , (
      SELECT B.EMP_NO
            , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO
      ) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY A.DEPT_CD
ORDER BY A.DEPT_CD
;
```

❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함

부서코드	부서명	직원수	연봉합계	평균연봉
100001	운영본부	1	₩46,270,000	₩46,270,000
100002	지원팀	4	₩201,290,000	₩50,322,500
100003	기획팀	4	₩201,870,000	₩50,467,500
100004	디자인팀	4	₩197,450,000	₩49,362,500
100005	플랫폼사업본부	1	₩58,690,000	₩58,690,000
100006	데이터팀	4	₩204,700,000	₩51,175,000
100007	개발팀	4	₩198,190,000	₩49,547,500
100008	솔루션사업본부	1	₩46,430,000	₩46,430,000
100009	운영팀	4	₩203,040,000	₩50,760,000
100010	개발팀	4	₩199,010,000	₩49,752,500
100011	신사업본부	1	₩53,490,000	₩53,490,000
100012	인공지능팀	4	₩212,340,000	₩53,085,000
100013	빅데이터팀	4	₩211,820,000	₩52,955,000
999999	회장실	1	₩44,330,000	₩44,330,000

➤ 그룹 함수 실습 - 합계 데이터 출력 - ROLLUP 함수 사용

```
SELECT A.DEPT_CD "부서코드"
      , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.DEPT_CD) AS "부서명"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
      , (
      SELECT B.EMP_NO
            , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO
      ) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY ROLLUP(A.DEPT_CD)
ORDER BY A.DEPT_CD
;
```

- ❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함
- ❖ ROLLUP 함수를 이용하여 전체 직원수, 연봉합계, 평균연봉 까지도 출력함

부서코드	부서명	직원수	연봉합계	평균연봉
100001	운영본부	1	?46,270,000	?46,270,000
100002	지원팀	4	?201,290,000	?50,322,500
100003	기획팀	4	?201,870,000	?50,467,500
100004	디자인팀	4	?197,450,000	?49,362,500
100005	플랫폼사업본부	1	?58,690,000	?58,690,000
100006	데이터팀	4	?204,700,000	?51,175,000
100007	개발팀	4	?198,190,000	?49,547,500
100008	솔루션사업본부	1	?46,430,000	?46,430,000
100009	운영팀	4	?203,040,000	?50,760,000
100010	개발팀	4	?199,010,000	?49,752,500
100011	신사업본부	1	?53,490,000	?53,490,000
100012	인공지능팀	4	?212,340,000	?53,085,000
100013	빅데이터팀	4	?211,820,000	?52,955,000
999999	회장실	1	?44,330,000	?44,330,000
(NULL)	(NULL)	41	?2,078,920,000	?50,705,365

➤ 그룹 함수 실습 - 합계 데이터 출력 - ROLLUP함수 사용 - 인자 값 추가

```
SELECT A.DEPT_CD "부서코드"
      , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.DEPT_CD) AS "부서명"
      , A.SEX_CD AS "성별코드"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
      , (
      SELECT B.EMP_NO
            , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO
      ) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY ROLLUP(A.DEPT_CD,A.SEX_CD)
ORDER BY A.DEPT_CD, A.SEX_CD
;
```

- ❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함
- ❖ ROLLUP함수를 이용하여 전체 직원수, 연봉합계, 평균연봉 까지도 출력함
- ❖ ROLLUP함수를 이용하여 부서성별별 직원수, 연봉합계, 평균 연봉 까지도 출력함

부서코드	부서명	성별코드	직원수	연봉합계	평균연봉
100001	운영본부	1	1	₩46,270,000	₩46,270,000
100001	운영본부	(NULL)	1	₩46,270,000	₩46,270,000
100002	지원팀	1	3	₩148,490,000	₩49,496,666
100002	지원팀	2	1	₩52,800,000	₩52,800,000
100002	지원팀	(NULL)	4	₩201,290,000	₩50,322,500
100003	기획팀	1	1	₩48,800,000	₩48,800,000
100003	기획팀	2	3	₩153,070,000	₩51,023,333
100003	기획팀	(NULL)	4	₩201,870,000	₩50,467,500
100004	디자인팀	1	1	₩48,990,000	₩48,990,000
100004	디자인팀	2	3	₩148,460,000	₩49,486,666
100004	디자인팀	(NULL)	4	₩197,450,000	₩49,362,500
100005	플랫폼사업본부	1	1	₩58,690,000	₩58,690,000
100005	플랫폼사업본부	(NULL)	1	₩58,690,000	₩58,690,000
100006	데이터팀	1	2	₩98,420,000	₩49,210,000
100006	데이터팀	2	2	₩106,280,000	₩53,140,000
100006	데이터팀	(NULL)	4	₩204,700,000	₩51,175,000
100007	개발팀	1	1	₩46,780,000	₩46,780,000
100007	개발팀	2	3	₩151,410,000	₩50,470,000
100007	개발팀	(NULL)	4	₩198,190,000	₩49,547,500
100008	솔루션사업본부	1	1	₩46,430,000	₩46,430,000
100008	솔루션사업본부	(NULL)	1	₩46,430,000	₩46,430,000
100009	운영팀	1	2	₩97,920,000	₩48,960,000
100009	운영팀	2	2	₩105,120,000	₩52,560,000
100009	운영팀	(NULL)	4	₩203,040,000	₩50,760,000
100010	개발팀	1	1	₩50,210,000	₩50,210,000
100010	개발팀	2	3	₩148,800,000	₩49,600,000
100010	개발팀	(NULL)	4	₩199,010,000	₩49,752,500
100011	신사업본부	1	1	₩53,490,000	₩53,490,000
100011	신사업본부	(NULL)	1	₩53,490,000	₩53,490,000
100012	인공지능팀	1	2	₩99,760,000	₩49,880,000
100012	인공지능팀	2	2	₩112,580,000	₩56,290,000
100012	인공지능팀	(NULL)	4	₩212,340,000	₩53,085,000
100013	빅데이터팀	1	1	₩51,000,000	₩51,000,000
100013	빅데이터팀	2	3	₩160,820,000	₩53,606,666
100013	빅데이터팀	(NULL)	4	₩211,820,000	₩52,955,000
999999	회장실	1	1	₩44,330,000	₩44,330,000
999999	회장실	(NULL)	1	₩44,330,000	₩44,330,000
(NULL)	(NULL)	(NULL)	41	₩2,078,920,000	₩50,705,365

➤ 그룹 함수 실습 - 집계 데이터 출력 - ROLLUP함수 사용 - 인자 값 추가 - GROUPING 함수

```
SELECT CASE GROUPING(A.DEPT_CD) WHEN 1 THEN '모든부서' ELSE A.DEPT_CD END AS "부서코드"
      , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.DEPT_CD) AS "부서명"
      , CASE GROUPING(A.SEX_CD) WHEN 1 THEN '모든성별' ELSE A.SEX_CD END AS "성별코드"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
      , (
      SELECT B.EMP_NO
      , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO
      ) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY ROLLUP(A.DEPT_CD,A.SEX_CD)
ORDER BY A.DEPT_CD, A.SEX_CD
;
```

- ❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함
- ❖ ROLLUP함수를 이용하여 전체 직원수, 연봉합계, 평균연봉 까지도 출력함
- ❖ ROLLUP함수를 이용하여 부서성별별 직원수, 연봉합계, 평균연봉 까지도 출력함
- ❖ GROUPING 함수를 이용하여 모든성별, 모든부서라고 표기함

부서코드	부서명	성별코드	직원수	연봉합계	평균연봉
100001	운영본부	1	1	₩46,270,000	₩46,270,000
100001	운영본부	모든성별	1	₩46,270,000	₩46,270,000
100002	지원팀	1	3	₩148,490,000	₩49,496,666
100002	지원팀	2	1	₩52,800,000	₩52,800,000
100002	지원팀	모든성별	4	₩201,290,000	₩50,322,500
100003	기획팀	1	1	₩48,800,000	₩48,800,000
100003	기획팀	2	3	₩153,070,000	₩51,023,333
100003	기획팀	모든성별	4	₩201,870,000	₩50,467,500
100004	디자인팀	1	1	₩48,990,000	₩48,990,000
100004	디자인팀	2	3	₩148,460,000	₩49,486,666
100004	디자인팀	모든성별	4	₩197,450,000	₩49,362,500
100005	플랫폼사업본부	1	1	₩58,690,000	₩58,690,000
100005	플랫폼사업본부	모든성별	1	₩58,690,000	₩58,690,000
100006	데이터팀	1	2	₩98,420,000	₩49,210,000
100006	데이터팀	2	2	₩106,280,000	₩53,140,000
100006	데이터팀	모든성별	4	₩204,700,000	₩51,175,000
100007	개발팀	1	1	₩46,780,000	₩46,780,000
100007	개발팀	2	3	₩151,410,000	₩50,470,000
100007	개발팀	모든성별	4	₩198,190,000	₩49,547,500
100008	솔루션사업본부	1	1	₩46,430,000	₩46,430,000
100008	솔루션사업본부	모든성별	1	₩46,430,000	₩46,430,000
100009	운영팀	1	2	₩97,920,000	₩48,960,000
100009	운영팀	2	2	₩105,120,000	₩52,560,000
100009	운영팀	모든성별	4	₩203,040,000	₩50,760,000
100010	개발팀	1	1	₩50,210,000	₩50,210,000
100010	개발팀	2	3	₩148,800,000	₩49,600,000
100010	개발팀	모든성별	4	₩199,010,000	₩49,752,500
100011	신사업본부	1	1	₩53,490,000	₩53,490,000
100011	신사업본부	모든성별	1	₩53,490,000	₩53,490,000
100012	인공지능팀	1	2	₩99,760,000	₩49,880,000
100012	인공지능팀	2	2	₩112,580,000	₩56,290,000
100012	인공지능팀	모든성별	4	₩212,340,000	₩53,085,000
100013	빅데이터팀	1	1	₩51,000,000	₩51,000,000
100013	빅데이터팀	2	3	₩160,820,000	₩53,606,666
100013	빅데이터팀	모든성별	4	₩211,820,000	₩52,955,000
999999	회장실	1	1	₩44,330,000	₩44,330,000
999999	회장실	모든성별	1	₩44,330,000	₩44,330,000
모든부서	(NULL)	모든성별	41	₩2,078,920,000	₩50,705,365

➤ 그룹 함수 실습 - 합계 데이터 출력 - CUBE함수 사용

```
SELECT A.DEPT_CD "부서코드"
      , (SELECT L.DEPT_NM
          FROM TB_DEPT L
          WHERE L.DEPT_CD = A.DEPT_CD) AS "부서명"
      , A.SEX_CD AS "성별코드"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
      , (
          SELECT B.EMP_NO
                , SUM(B.PAY_AMT) AS "연봉"
          FROM TB_SAL_HIS B
          WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
          GROUP BY B.EMP_NO
          ORDER BY B.EMP_NO
        ) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY CUBE(A.DEPT_CD, A.SEX_CD)
ORDER BY A.DEPT_CD ;
```

- ❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함
- ❖ CUBE함수를 사용함으로써 다차원 집계를 구함
- ❖ 전체, 부서별, 부서성별별, 성별별, 합계를 구함 (CUBE함수의 인자 컬럼의 2개임, 2의 2승 =4)

부서코드	부서명	성별코드	직원수	연봉합계	평균연봉
100001	운영본부	1	1	₩46,270,000	₩46,270,000
100001	운영본부	(NULL)	1	₩46,270,000	₩46,270,000
100002	지원팀	1	3	₩148,490,000	₩49,496,666
100002	지원팀	2	1	₩52,800,000	₩52,800,000
100002	지원팀	(NULL)	4	₩201,290,000	₩50,322,500
100003	기획팀	1	1	₩48,800,000	₩48,800,000
100003	기획팀	2	3	₩153,070,000	₩51,023,333
100003	기획팀	(NULL)	4	₩201,870,000	₩50,467,500
100004	디자인팀	1	1	₩48,990,000	₩48,990,000
100004	디자인팀	2	3	₩148,460,000	₩49,486,666
100004	디자인팀	(NULL)	4	₩197,450,000	₩49,362,500
100005	플랫폼사업본부	1	1	₩58,690,000	₩58,690,000
100005	플랫폼사업본부	(NULL)	1	₩58,690,000	₩58,690,000
100006	데이터팀	1	2	₩98,420,000	₩49,210,000
100006	데이터팀	2	2	₩106,280,000	₩53,140,000
100006	데이터팀	(NULL)	4	₩204,700,000	₩51,175,000
100007	개발팀	1	1	₩46,780,000	₩46,780,000
100007	개발팀	2	3	₩151,410,000	₩50,470,000
100007	개발팀	(NULL)	4	₩198,190,000	₩49,547,500
100008	솔루션사업본부	1	1	₩46,430,000	₩46,430,000
100008	솔루션사업본부	(NULL)	1	₩46,430,000	₩46,430,000
100009	운영팀	1	2	₩97,920,000	₩48,960,000
100009	운영팀	2	2	₩105,120,000	₩52,560,000
100009	운영팀	(NULL)	4	₩203,040,000	₩50,760,000
100010	개발팀	1	1	₩50,210,000	₩50,210,000
100010	개발팀	2	3	₩148,800,000	₩49,600,000
100010	개발팀	(NULL)	4	₩199,010,000	₩49,752,500
100011	신사업본부	1	1	₩53,490,000	₩53,490,000
100011	신사업본부	(NULL)	1	₩53,490,000	₩53,490,000
100012	인공지능팀	1	2	₩99,760,000	₩49,880,000
100012	인공지능팀	2	2	₩112,580,000	₩56,290,000
100012	인공지능팀	(NULL)	4	₩212,340,000	₩53,085,000
100013	빅데이터팀	1	1	₩51,000,000	₩51,000,000
100013	빅데이터팀	2	3	₩160,820,000	₩53,606,666
100013	빅데이터팀	(NULL)	4	₩211,820,000	₩52,955,000
999999	회장실	1	1	₩44,330,000	₩44,330,000
999999	회장실	(NULL)	1	₩44,330,000	₩44,330,000
(NULL)	(NULL)	1	19	₩939,580,000	₩49,451,578
(NULL)	(NULL)	2	22	₩1,139,340,000	₩51,788,181
(NULL)	(NULL)	(NULL)	41	₩2,078,920,000	₩50,705,365

➤ 그룹 함수 실습 - 합계 데이터 출력 - UNION ALL & GROUP BY

```
SELECT A.DEPT_CD "부서코드", '모든성별' AS "성별코드"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
      , (
      SELECT B.EMP_NO
            , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY A.DEPT_CD
UNION ALL
SELECT '모든부서' AS "부서코드", A.SEX_CD AS "성별코드"
      , COUNT(*) AS "부서별직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "부서별연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "부서별평균연봉"
FROM TB_EMP A
      , (
      SELECT B.EMP_NO
            , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY A.SEX_CD
ORDER BY "부서코드", "성별코드";
```

- ❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함
- ❖ UNION ALL 및 GROUP BY를 이용하여 부서별, 성별 별 인원수와, 연봉합계, 평균연봉을 출력함

부서코드	성별코드	직원수	연봉합계	평균연봉
100001	모든성별	1	₩46,270,000	₩46,270,000
100002	모든성별	4	₩201,290,000	₩50,322,500
100003	모든성별	4	₩201,870,000	₩50,467,500
100004	모든성별	4	₩197,450,000	₩49,362,500
100005	모든성별	1	₩58,690,000	₩58,690,000
100006	모든성별	4	₩204,700,000	₩51,175,000
100007	모든성별	4	₩198,190,000	₩49,547,500
100008	모든성별	1	₩46,430,000	₩46,430,000
100009	모든성별	4	₩203,040,000	₩50,760,000
100010	모든성별	4	₩199,010,000	₩49,752,500
100011	모든성별	1	₩53,490,000	₩53,490,000
100012	모든성별	4	₩212,340,000	₩53,085,000
100013	모든성별	4	₩211,820,000	₩52,955,000
999999	모든성별	1	₩44,330,000	₩44,330,000
모든부서	1	19	₩939,580,000	₩49,451,578
모든부서	2	22	₩1,139,340,000	₩51,788,181

➤ 그룹 함수 실습 - 집계 데이터 출력 - GROUPING SETS

```
SELECT DECODE(GROUPING(A.DEPT_CD), 1, '모든부서', A.DEPT_CD) AS "부서코드"
      , DECODE(GROUPING(A.SEX_CD), 1, '모든성별', A.SEX_CD) AS "성별코드"
      , COUNT(*) AS "직원수"
      , TO_CHAR(TRUNC(SUM(B."연봉")), 'L999,999,999,999') AS "연봉합계"
      , TO_CHAR(TRUNC(AVG(B."연봉")), 'L999,999,999,999') AS "평균연봉"
FROM TB_EMP A
    , (
      SELECT B.EMP_NO
            , SUM(B.PAY_AMT) AS "연봉"
      FROM TB_SAL_HIS B
      WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
      GROUP BY B.EMP_NO
      ORDER BY B.EMP_NO
    ) B
WHERE A.EMP_NO = B.EMP_NO
GROUP BY GROUPING SETS(A.DEPT_CD, A.SEX_CD)
ORDER BY "부서코드", "성별코드"
;
```

- ❖ 2019년도 기준 부서별 직원수, 연봉합계, 평균연봉을 출력함
- ❖ GROUPING SETS 함수를 이용하여 부서별, 성별별 인원수와, 연봉합계, 평균연봉을 출력함
- ❖ GROUPING 함수를 이용하여 모든부서, 모든성별을 출력한다.

부서코드	성별코드	직원수	연봉합계	평균연봉
100001	모든성별	1	₩46,270,000	₩46,270,000
100002	모든성별	4	₩201,290,000	₩50,322,500
100003	모든성별	4	₩201,870,000	₩50,467,500
100004	모든성별	4	₩197,450,000	₩49,362,500
100005	모든성별	1	₩58,690,000	₩58,690,000
100006	모든성별	4	₩204,700,000	₩51,175,000
100007	모든성별	4	₩198,190,000	₩49,547,500
100008	모든성별	1	₩46,430,000	₩46,430,000
100009	모든성별	4	₩203,040,000	₩50,760,000
100010	모든성별	4	₩199,010,000	₩49,752,500
100011	모든성별	1	₩53,490,000	₩53,490,000
100012	모든성별	4	₩212,340,000	₩53,085,000
100013	모든성별	4	₩211,820,000	₩52,955,000
999999	모든성별	1	₩44,330,000	₩44,330,000
모든부서	1	19	₩939,580,000	₩49,451,578
모든부서	2	22	₩1,139,340,000	₩51,788,181

Chapter 04. SQL 활용

4-6. 윈도우 함수 (WINDOW FUNCTION)

➤ 윈도우 함수 개요

- ① 행과 행간의 관계에서 다양한 연산 처리를 할 수 있는 함수가 윈도우 함수이다.
- ② 분석함수로도 알려져 있다. (ANSI 표준은 윈도우 함수이다.)
- ③ 윈도우함수는 일반 함수와 달리 중첩하여 호출 될 수는 없다.

➤ 윈도우 함수의 종류

종류	설명
순위관련함수	• RANK • DENSE_RANK • ROW_NUMBER
집계관련함수	• SUM • MAX • MIN • AVG • COUNT
행순서관련함수	• FIRST_VALUE • LAST_VALUE • LAG • LEAD
그룹내 비율관련함수	• CUME_DIST • PERCENT_RANK • NTILE • RATIO_TO_REPORT

➤ 윈도우 함수 문법

```
SELECT
    윈도우함수(인자) OVER (PARTITION BY 컬럼 ORDER BY 컬럼)
    윈도우절(ROWS|RANGE BETWEEN UNBOUND PRECEDING|CURRENT ROW AND UNBOUNDED FOLLOWING|CURRENT ROW)
FROM 테이블명
;
```

항목	설명
윈도우함수	• 다양한 윈도우 함수를 사용 가능
인자	• 함수에 따라 0~N개의 인자를 사용
PARTITION BY	• FROM절 이하에서 나온 결과집합을 특정 컬럼(들)을 기준으로 그룹화할 수 있다. 예를 전체 직원정보를 출력 하면서 각 직원이 속한 부서별로 그룹화할 수 있다)
ORDER BY	• ORDER BY에 기재한 정렬 기준에 윈도우 함수의 결과가 달라질 수 있다. • 예를 들어 RANK 함수를 이용시 ORDER BY에 연봉을 DESC로 기재하면 연봉이 높은 사람이 1등이 된다.
윈도우절	• 윈도우 함수가 연산을 처리하는 대상이 되는 행의 범위를 지정할 수 있다. • ROWS는 물리적인 결과행의 수를 뜻한다. • RANGE는 논리적인 값에 의한 범위를 뜻한다. • BETWEEN ~ AND는 행 범위의 시작과 끝을 지정하는데 사용된다. • UNBOUNDED PRECEDING는 행 범위의 시작 위치가 전체 행 범위에서 첫 번째 행임을 뜻한다. • UNBOUNDED FOLLOWING는 행 범위의 마지막 위치가 전체 행 범위에서 마지막 행임을 뜻한다. • CURRENT ROW는 행 범위의 시작 위치가 현재 행임을 뜻한다. • 1 PRECEDING은 행 범위의 시작위치가 1만큼 이전 행임을 의미한다. • 1 FOLLOWING은 행 범위의 마지막위치가 1만큼 다음 행임을 의미한다

➤ 윈도우 함수 실습 - 그룹내순위함수

```
SELECT A.EMP_NO, A.EMP_NM, A.BIRTH_DE, A.DEPT_CD
      , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.DEPT_CD) AS DEPT_NM
      , RANK() OVER(ORDER BY A.BIRTH_DE) AS RANK
      , DENSE_RANK() OVER(ORDER BY A.BIRTH_DE) AS DENSE_RANK
      , ROW_NUMBER() OVER(ORDER BY A.BIRTH_DE) AS ROW_NUMBER
      , RANK() OVER(PARTITION BY A.DEPT_CD ORDER BY A.BIRTH_DE) AS RANK_DEPT_CD
FROM TB_EMP A
WHERE A.SEX_CD = '1' --남성
ORDER BY A.BIRTH_DE;
```

- ❖ 전 직원중 성별이 남성이 직원들의 생년월일을 출력하고 생년월일 순으로 RANK를 구함
- ❖ RANK 함수는 동일값이라면 동일 순위라고 판단함 1 2 2 4 순으로 순위를 정함
- ❖ DENSE_RANK 함수는 동일값이라면 동일 순위라고 판단함 1 2 2 3 순으로 순위를 정함
- ❖ ROW_NUMBER 함수는 동일값이라도 무조건 순위를 정함
- ❖ DEPT_CD기준으로 PARTITION BY하여 부서별 생일 순위도 같이 구하였음

EMP_NO	EMP_NM	BIRTH_DE	DEPT_CD	DEPT_NM	RANK	DENSE_RANK	ROW_NUMBER	RANK_DEPT_CD
999999999	김 회장	19651105	999999	회장실	1	1	1	1
100000026	김길정	19690524	100009	운영팀	2	2	2	1
100000035	최창수	19690524	100012	인공지능팀	2	2	3	1
100000032	이준표	19771202	100011	신사업본부	4	3	4	1
100000014	이관심	19780213	100005	플랫폼사업본부	5	4	5	1
100000023	이관심	19780213	100008	솔루션사업본부	5	4	6	1
100000001	이경오	19840612	100001	운영본부	7	5	7	1
100000024	박선영	19870615	100009	운영팀	8	6	8	2
100000033	홍사기	19870615	100012	인공지능팀	8	6	9	2
100000004	이승준	19870623	100002	지원팀	10	7	10	1
100000017	이경손	19880124	100006	데이터팀	11	8	11	1
100000009	박태범	19880629	100003	기획팀	12	9	12	1
100000002	이현승	19880705	100002	지원팀	13	10	13	2
100000012	김호형	19910227	100004	디자인팀	14	11	14	1
100000021	김열호	19910227	100007	개발팀	14	11	15	1
100000003	이정수	19911224	100002	지원팀	16	12	16	3
100000039	이박력	19940709	100013	빅데이터팀	17	13	17	1
100000030	이규호	19970627	100010	개발팀	18	14	18	1
100000015	이정직	19980715	100006	데이터팀	19	15	19	2

➤ 윈도우 함수 실습 - 집계관련함수

```

SELECT A.EMP_NO
      , A.MAX_EMP_NM
      , A.연봉
      , A.MAX_DEPT_CD
      , (SELECT DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.MAX_DEPT_CD) AS DEPT_NM
      , SUM(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD) AS "속한부서의연봉총액"
      , SUM(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD ORDER BY A.연봉
                          RANGE UNBOUNDED PRECEDING) AS "속한부서의연봉누적합계_RANGE"
      , SUM(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD ORDER BY A.연봉
                          ROWS UNBOUNDED PRECEDING) AS "속한부서의연봉누적합계_ROWS"
      , MAX(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD) AS "속한부서의최고연봉액"
      , MIN(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD) AS "속한부서의최저연봉액"
      , TRUNC(AVG(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD)) AS "속한부서의평균연봉액"
      , TRUNC(AVG(A.연봉) OVER(PARTITION BY A.MAX_DEPT_CD ORDER BY A.연봉
                              ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
                              )
              ) AS "속한부서에서앞뒤자신의평균연봉액"
      , COUNT(*) OVER (PARTITION BY A.MAX_DEPT_CD) AS "부서별직원수"
FROM
(
  SELECT B.EMP_NO
        , MAX(A.EMP_NM) AS MAX_EMP_NM
        , MAX(A.DEPT_CD) AS MAX_DEPT_CD
        , SUM(B.PAY_AMT) AS "연봉"
    FROM TB_SAL_HIS B , TB_EMP A
   WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
   AND A.EMP_NO = B.EMP_NO
   GROUP BY B.EMP_NO
   ORDER BY B.EMP_NO
) A
ORDER BY A.MAX_DEPT_CD, A.연봉;

```

```

SELECT * FROM TB_SAL_HIS
WHERE EMP_NO = '1000000003'
AND PAY_DE = '20191225'
;

UPDATE TB_SAL_HIS
SET PAY_AMT = PAY_AMT + 3480000
WHERE EMP_NO = '1000000003'
AND PAY_DE = '20191225'

```

- ❖ 2019년 기준 전직원의 연봉액수를 구하고
- ❖ 자신이 속한 부서의 연봉총액이 얼마인지를 구함
- ❖ 자신이 속한 부서의 연봉총액이 얼마인지를 구하는데 누적합계로 보여줌
- ❖ 자신이 속한 부서의 최고연봉액수를 보여줌
- ❖ 자신이 속한 부서의 최저연봉액수를 보여줌
- ❖ 자신이 속한 부서의 평균연봉액수를 보여줌
- ❖ 자신이 속한 부서의 앞자신뒤의평균연봉액수를 보여줌
- ❖ 자신이 속한 부서의 직원수를 보여줌

➤ 윈도우 함수 실습 - 집계관련함수

EMP_NO	MAX_EMP_NM	연봉	MAX_DEPT_CD	DEPT_NM	속한부서의 연봉총액	속한부서의연봉누적합계 _RANGE	속한부서의연봉누적합계 _ROWS	속한부서의 최고연봉액	속한부서의 최저연봉액	속한부서의 평균연봉액	속한부서에서 앞뒤자신의평균연봉액	부서별 직원수
1000000001	이경오	46270000	100001	운영본부	46270000	46270000	46270000	46270000	46270000	46270000	46270000	1
1000000003	이정수	45740000	100002	지원팀	201290000	45740000	45740000	53530000	45740000	50322500	47480000	4
1000000004	이승준	49220000	100002	지원팀	201290000	94960000	94960000	53530000	45740000	50322500	49253333	4
1000000005	김현희	52800000	100002	지원팀	201290000	147760000	147760000	53530000	45740000	50322500	51850000	4
1000000002	이현승	53530000	100002	지원팀	201290000	201290000	201290000	53530000	45740000	50322500	53165000	4
중간생략...												
1000000039	이박력	51000000	100013	빅데이터팀	211820000	51000000	51000000	54220000	51000000	52955000	51850000	4
1000000037	이현정	52700000	100013	빅데이터팀	211820000	103700000	103700000	54220000	51000000	52955000	52533333	4
1000000038	김혜수	53900000	100013	빅데이터팀	211820000	157600000	157600000	54220000	51000000	52955000	53606666	4
1000000040	김여진	54220000	100013	빅데이터팀	211820000	211820000	211820000	54220000	51000000	52955000	54060000	4
9999999999	김회장	44330000	999999	회장실	44330000	44330000	44330000	44330000	44330000	44330000	44330000	1

➤ 윈도우 함수 실습 - 행순서관련함수 - 실습 환경 구축

```
DROP TABLE TB_REAL_IDX PURGE;

CREATE TABLE TB_REAL_IDX
(
  SEQ NUMBER(15)
, SECTOR_NM VARCHAR2(50)
, STD_DE CHAR(8)
, STD_TM CHAR(6)
, CUR_IDX NUMBER(15, 2)
, CONSTRAINT PK_TB_REAL_IDX
  PRIMARY KEY(SEQ)
);
```

```
INSERT INTO TB_REAL_IDX
SELECT ROWNUM AS RNUM
, '코스피' AS SECTOR_NM
, '20200629' AS STD_DE
, TO_CHAR(TO_DATE('090000', 'HH24MISS') + (ROWNUM*60)/24/60/60, 'HH24MISS') AS HH24MISS
, CUR_IDX
FROM
(
  SELECT
    ROUND(DBMS_RANDOM.VALUE(2000.00, 2050.99), 2) AS CUR_IDX
  FROM DUAL CONNECT BY LEVEL <= 390
  ORDER BY CUR_IDX
)
UNION ALL
SELECT ROWNUM+390 AS RNUM
, '코스닥' AS SECTOR_NM
, '20200629' AS STD_DE
, TO_CHAR(TO_DATE('090000', 'HH24MISS') + (ROWNUM*60)/24/60/60, 'HH24MISS') AS HH24MISS
, CUR_IDX
FROM
(
  SELECT
    ROUND(DBMS_RANDOM.VALUE(700.00, 725.99), 2) AS CUR_IDX
  FROM DUAL CONNECT BY LEVEL <= 390
  ORDER BY CUR_IDX
)
;

COMMIT;
```

```
SELECT * FROM TB_REAL_IDX ORDER BY SEQ;
```

➤ 윈도우 함수 실습 - 행순서관련함수

```
SELECT A.SEQ
      , A.SECTOR_NM
      , A.STD_DE
      , A.STD_TM
      , A.CUR_IDX
      , FIRST_VALUE(CUR_IDX) OVER(PARTITION BY A.SECTOR_NM ORDER BY A.STD_TM
                                   ROWS UNBOUNDED PRECEDING) AS "각지수의첫지수값"
      , LAST_VALUE(CUR_IDX) OVER(PARTITION BY A.SECTOR_NM ORDER BY A.STD_TM
                                   ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING) AS "각지수의마지막지수값"
      , LAG(CUR_IDX, 1) OVER (PARTITION BY A.SECTOR_NM ORDER BY A.STD_TM) AS "이전시간의지수값"
      , LEAD(CUR_IDX, 1) OVER (PARTITION BY A.SECTOR_NM ORDER BY A.STD_TM) AS "다음시간의지수값"
FROM TB_REAL_IDX A
ORDER BY A.SECTOR_NM DESC, A.STD_DE, A.STD_TM
;
```

- ❖ ROWS UNBOUNDED PRECEDING : 현재행을 기준으로 파티션내의 첫번째 행까지의 범위 지정
- ❖ ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING : 현재행을 기준으로 파티션내의 마지막 행까지의 범위 지정

SEQ	SECTOR_NM	STD_DE	STD_TM	CUR_IDX	각지수의첫지수값	각지수의마지막지수값	이전시간의지수값	다음시간의지수값
1	코스피	20200629	90100	2000.11	2000.11	2050.72	(NULL)	2000.29
2	코스피	20200629	90200	2000.29	2000.11	2050.72	2000.11	2000.31
3	코스피	20200629	90300	2000.31	2000.11	2050.72	2000.29	2000.57
중간생략...								
388	코스피	20200629	152800	2050.55	2000.11	2050.72	2050.36	2050.57
389	코스피	20200629	152900	2050.57	2000.11	2050.72	2050.55	2050.72
390	코스피	20200629	153000	2050.72	2000.11	2050.72	2050.57	(NULL)
391	코스닥	20200629	90100	700.02	700.02	725.81	(NULL)	700.04
392	코스닥	20200629	90200	700.04	700.02	725.81	700.02	700.18
393	코스닥	20200629	90300	700.18	700.02	725.81	700.04	700.2
394	코스닥	20200629	90400	700.2	700.02	725.81	700.18	700.26
중간생략...								
395	코스닥	20200629	90500	700.26	700.02	725.81	700.2	700.31
777	코스닥	20200629	152700	725.57	700.02	725.81	725.57	725.58
778	코스닥	20200629	152800	725.58	700.02	725.81	725.57	725.71
779	코스닥	20200629	152900	725.71	700.02	725.81	725.58	725.81
780	코스닥	20200629	153000	725.81	700.02	725.81	725.71	(NULL)

➤ 윈도우 함수 실습 - 그룹내 비율관련함수

```

SELECT
    A.MAX_DEPT_CD
  , A.부서별연봉총액
  , (SELECT L.DEPT_NM FROM TB_DEPT L WHERE L.DEPT_CD = A.MAX_DEPT_CD) AS DEPT_NM
  , ROUND(RATIO_TO_REPORT(A.부서별연봉총액) OVER(), 4) * 100 || '%' AS "부서별연봉비율"
  , ROUND(PERCENT_RANK() OVER(ORDER BY A.부서별연봉총액), 4) *100 || '%' AS "부서별연봉비율순서별백분율"
  , ROUND(CUME_DIST() OVER(ORDER BY A.부서별연봉총액), 4) *100 || '%' AS "부서별연봉비율순서별누적백분율"
  , NTILE(4) OVER(ORDER BY A.부서별연봉총액) AS "부서별연봉비율순서별등분결과"

FROM
(
SELECT
    A.MAX_DEPT_CD
  , SUM(A.연봉) AS "부서별연봉총액"
FROM
(
SELECT B.EMP_NO
  , MAX(A.EMP_NM) AS MAX_EMP_NM
  , MAX(A.DEPT_CD) AS MAX_DEPT_CD
  , SUM(B.PAY_AMT) AS "연봉"
FROM TB_SAL_HIS B , TB_EMP A
WHERE B.PAY_DE BETWEEN '20190101' AND '20191231'
AND A.EMP_NO = B.EMP_NO
GROUP BY B.EMP_NO
ORDER BY B.EMP_NO
) A
GROUP BY A.MAX_DEPT_CD
ORDER BY A.MAX_DEPT_CD
) A
;

```

- ❖ RATIO_TO_REPORT 함수로 부서별 연봉의 비율을 구함
- ❖ PERCENT_RANK함수로 부서별연봉비율순서의 백분율을 구함
- ❖ CUME_DIST함수로 부서별연봉비율의 누적 백분율을 구함
- ❖ NTILE함수로 부서별연봉비율의 등분 결과를 구함

➤ 윈도우 함수 실습 - 그룹내 비율관련함수

M AX_DEPT_CD	부서별연봉총액	DEPT_NM	부서별연봉비율	부서별연봉비율순서별백분율	부서별연봉비율순서별누적백분율	부서별연봉비율순서별등분결과
999999	44330000	회장실	2.13%	0%	7.14%	1
100001	46270000	운영본부	2.23%	7.69%	14.29%	1
100008	46430000	솔루션사업본부	2.23%	15.38%	21.43%	1
100011	53490000	신사업본부	2.57%	23.08%	28.57%	1
100005	58690000	플랫폼사업본부	2.82%	30.77%	35.71%	2
100004	197450000	디자인팀	9.50%	38.46%	42.86%	2
100007	198190000	개발팀	9.53%	46.15%	50%	2
100010	199010000	개발팀	9.57%	53.85%	57.14%	2
100002	201290000	지원팀	9.68%	61.54%	64.29%	3
100003	201870000	기획팀	9.71%	69.23%	71.43%	3
100009	203040000	운영팀	9.77%	76.92%	78.57%	3
100006	204700000	데이터팀	9.85%	84.62%	85.71%	4
100013	211820000	빅데이터팀	10.19%	92.31%	92.86%	4
100012	212340000	인공지능팀	10.21%	100%	100%	4

Chapter 04. SQL 활용

4-7. DCL (DATA CONTROL LANGUAGE)

➤ DCL이란?

- ① 유저를 생성하고 권한을 제어할 수 있는 명령어
- ② 데이터의 보호와 보안을 위해서 유저와 권한을 관리 해야함

➤ 오라클에서 제공하는 유저들

유저	설명
SCOTT	• 테스트용 샘플 유저
SYS	• DBA 권한을 부여 받은 유저 (최상위 유저)
SYSTEM	• SYSTEM 데이터베이스의 모든 시스템 권한을 부여 받은 유저(SYS바로 밑)

➤ 유저 생성(SYSTEM계정으로 접속)

```
DROP USER SQLD_TEST CASCADE;  
CREATE USER SQLD_TEST IDENTIFIED BY 1234;
```

✓ DROP USER 부분에서 에러가 나도 상관없음 있으면 지우고 없으면 에러가 발생하는데 에러가 발생해도 CREATE USER로 넘어가도 됨

➤ SQLD_TEST로 접속 시도

```
ORA-01045: 사용자 SQLD_TEST는 CREATE SESSION 권한을 가지고 있지 않음, 로그인이 거절되었습니다.
```

❖ 에러 발생

➤ 접속 권한 주기(SYSTEM계정으로 접속)

```
GRANT CREATE SESSION TO SQLD_TEST;
```

➤ SQLD_TEST로 접속 시도(SQLD_TEST 계정으로 접속)

❖ SQLD_TEST로 접속 성공

```
CREATE TABLE TB_SQLD_TEST
(
  SQLD_TEST_NO CHAR(10)
, SQLD_TEST_NM VARCHAR2(50)
, CONSTRAINT PK_TB_SQLD_TEST PRIMARY KEY(SQLD_TEST_NO)
)
;
```

❖ 테이블 생성 실패

ORA-01031: 권한이 불충분합니다

➤ 테이블 생성 권한 주기(SYSTEM계정으로 접속)

```
GRANT CREATE TABLE TO SQLD_TEST;
```

➤ 테이블 생성 재시도(SQLD_TEST 계정으로 접속)

```
CREATE TABLE TB_SQLD_TEST
(
  SQLD_TEST_NO CHAR(10)
, SQLD_TEST_NM VARCHAR2(50)
, CONSTRAINT PK_TB_SQLD_TEST PRIMARY KEY(SQLD_TEST_NO)
)
;
```

❖ 테이블 생성 성공

```
SELECT * FROM TB_SQLD_TEST;
```

❖ 테이블 조회 성공

➤ **SQLD_TEST로 SQLD 유저의 테이블 조회 시도(SQLD_TEST 계정으로 접속)**

```
SELECT * FROM SQLD.TB_EMP;
```

 ❖ 테이블 조회 실패

ORA-00942: 테이블 또는 뷰가 존재하지 않습니다

➤ **테이블 조회 권한 주기(SYSTEM계정으로 접속)**

```
GRANT SELECT ON SQLD.TB_EMP TO SQLD_TEST;
```

➤ **SQLD_TEST로 SQLD 유저의 테이블 조회 재시도(SQLD_TEST 계정으로 접속)**

```
SELECT * FROM SQLD.TB_EMP;
```

 ❖ 테이블 조회 성공

➤ **테이블 DML 권한 주기(SYSTEM계정으로 접속)**

```
GRANT INSERT, DELETE, UPDATE ON SQLD.TB_EMP TO SQLD_TEST;
```

- ❖ INSERT, DELETE, UPDATE 권한도 줌
- ❖ SQLD_TEST 계정은 해당 테이블에 대한 INSERT, DELETE, UPDATE도 수행할 수 있게 됨

➤ ROLE을 이용한 권한 부여

- ① 유저를 생성하면 다양한 많은 권한들을 부여 해야함
- ② DBA는 ROLE을 생성하고 ROLE에 각종 권한을 부여한 후 해당 ROLE을 다른 유저에게 부여
- ③ ROLE에 포함된 권한들이 필요한 유저에게 빠르게 권한을 부여할 수 있음

➤ 테이블 생성 권한 회수(SYSTEM계정으로 접속)

```
REVOKE CREATE TABLE FROM SQLD_TEST;
```

❖ SQLD_TEST가 가진 CREATE TABLE 권한 회수

➤ SQLD_TEST로 테이블 생성 시도 (SQLD_TEST 계정으로 접속)

```
CREATE TABLE TB_SQLD_TEST_2
(
  SQLD_TEST_NO CHAR(10)
, SQLD_TEST_NM VARCHAR2(50)
, CONSTRAINT PK_TB_SQLD_TEST_2 PRIMARY KEY(SQLD_TEST_NO)
)
;
```

❖ 테이블 생성 실패

ORA-01031: 권한이 불충분합니다

➤ 롤을 생성한 후 SQL_TEST에게 롤 권한 부여(SYSTEM계정으로 접속)

```
DROP ROLE CREATE_SESSION_TABLE;
CREATE ROLE CREATE_SESSION_TABLE;
GRANT CREATE SESSION, CREATE TABLE TO CREATE_SESSION_TABLE;
GRANT CREATE_SESSION_TABLE TO SQLD_TEST;
```

➤ SQLD_TEST로 테이블 생성 재시도 (SQLD_TEST 계정으로 접속)

```
CREATE TABLE TB_SQLD_TEST_2
(
  SQLD_TEST_NO CHAR(10)
, SQLD_TEST_NM VARCHAR2(50)
, CONSTRAINT PK_TB_SQLD_TEST_2 PRIMARY KEY(SQLD_TEST_NO)
)
;
```

❖ 테이블 생성 성공

➤ 오라클 DBMS에서 일반적으로 부여하는 ROLE

ROLE명	부여 권한
CONNECT	• CREATE SESSION
RESOURCE	• CREATE CLUSTER • CREATE PROCEDURE • CREATE TYPE • CREATE SEQUENCE • CREATE TRIGGER • CREATE OPERATOR • CREATE TABLE • CREATE INDEXTYPE

➤ SQLD_TEST유저 제거 및 생성, 기본 롤 부여(SYSTEM계정으로 접속)

```
DROP USER SQLD_TEST CASCADE; --생성한 테이블도 같이 제거됨
CREATE USER SQLD_TEST IDENTIFIED BY 1234;

GRANT CONNECT, RESOURCE TO SQLD_TEST;
```

➤ 테이블 생성 시도(SQLD_TEST 계정으로 접속)

```
DROP TABLE TB_SQLD_TEST;
```

```
CREATE TABLE TB_SQLD_TEST  
(  
    SQLD_TEST_NO CHAR(10)  
    , SQLD_TEST_NM VARCHAR2(50)  
    , CONSTRAINT PK_TB_SQLD_TEST PRIMARY KEY(SQLD_TEST_NO)  
)  
;
```

❖ 테이블 생성 성공

Chapter 04. SQL 활용

4-8. 절차형 SQL

➤ 절차형 SQL이란?

- ① 일반적인 개발언어처럼 SQL문도 절차지향적인 프로그램 작성이 가능하도록 절차형 SQL을 제공한다.
- ② 절차형 SQL을 사용하면 SQL문의 연속적인 실행이나 조건에 따른 분기 처리를 수행하는 모듈을 생성할 수 있다.
- ③ 오라클 기준 이러한 절차형 모듈의 종류는 프로시저, 사용자정의함수, 트리거가 있다.
- ④ 오라클 기준 이러한 절차형 모듈을 PL/SQL이라고 부른다.

➤ PL/SQL의 개요

- ① PL/SQL은 Block 구조로 되어 있고 Block 내에는 SQL문, IF, LOOP 등이 존재함
- ② PL/SQL을 이용해서 다양한 모듈을 개발 가능

➤ PL/SQL의 특징

- ① Block구조로 되어있으며 각 기능별로 모듈화가 가능
- ② 변수/상수 선언 및 IF/LOOP문 등의 사용이 가능
- ③ DBMS에러나 사용자 에러 정의를 할 수 있음
- ④ PL/SQL은 오라클에 내장 시킬수 있으므로 어떠한 오라클 서버로도 이식이 가능
- ⑤ PL/SQL은 여러 SQL문장을 Block으로 묶고 한번에 Block전부를 서버로 보내기때문에 네트워크 패킷 수를 감소 시킴

➤ PL/SQL Block의 구조

구조 명	필수/선택	설명
DECLARE(선언 부)	필수	• BEGIN~END에서 사용할 변수나 인수에 대한 정의 및 데이터 타입 선언
BEGIN(실행 부)	필수	• 개발자가 처리하고자 하는 SQL문과 필요한 LOGIC (비교 문, 제어 문 등)이 정의되는 실행 부
EXCEPTION(예외 처리 부)	선택	• BEGIN~END에서 실행되는 SQL문에 발생한 에러를 처리하는 에러 처리 부
END	필수	

❖ PL/SQL은 DECLARE, BEGIN, EXCEPTION, END로 이루어져 있으며 그중 EXCEPTION은 선택항목이고 나머지는 필수 항목이다.

➤ 오라클 프로시저 실습 준비

```
DROP TABLE TB_EMP_PAY_BY_YEAR;  
CREATE TABLE TB_EMP_PAY_BY_YEAR  
(  
  EMP_NO CHAR(10) NOT NULL  
, STD_YEAR CHAR(4) NOT NULL  
, PAY_AMT NUMBER(15) NOT NULL  
, CONSTRAINT PK_TB_EMP_PAY_BY_YEAR PRIMARY KEY(EMP_NO, STD_YEAR)  
)  
;
```

❖ "연별직원급여지급내역" 테이블을 생성한다.

➤ 오라클 프로시저 생성

```
CREATE OR REPLACE PROCEDURE SP_INSERT_TB_EMP_PAY_BY_YEAR
(IN_STD_YEAR IN TB_EMP_PAY_BY_YEAR.STD_YEAR%TYPE)
IS
V_EMP_NO TB_SAL_HIS.EMP_NO%TYPE;
V_SEX_CD TB_EMP.SEX_CD%TYPE;
V_STD_YEAR TB_EMP_PAY_BY_YEAR.STD_YEAR%TYPE;
V_PAY_AMT TB_SAL_HIS.PAY_AMT%TYPE;
CURSOR SELECT_TB_EMP IS
SELECT B.EMP_NO
      , (SELECT L.SEX_CD FROM TB_EMP L WHERE L.EMP_NO = B.EMP_NO) AS SEX_CD
      , SUBSTR(B.PAY_DE, 1, 4) AS STD_YEAR
      , SUM(B.PAY_AMT) AS PAY_AMT
FROM TB_SAL_HIS B
WHERE B.PAY_DE BETWEEN IN_STD_YEAR || '0101' AND IN_STD_YEAR || '1231'
GROUP BY B.EMP_NO, SUBSTR(B.PAY_DE, 1, 4)
ORDER BY B.EMP_NO;

BEGIN
OPEN SELECT_TB_EMP;
DBMS_OUTPUT.PUT_LINE('-----');
LOOP
FETCH SELECT_TB_EMP INTO V_EMP_NO, V_SEX_CD, V_STD_YEAR, V_PAY_AMT;
EXIT WHEN SELECT_TB_EMP%NOTFOUND;

DBMS_OUTPUT.PUT_LINE('V_EMP_NO      : ' || '[' || V_EMP_NO || ']');
DBMS_OUTPUT.PUT_LINE('V_SEX_CD      : ' || '[' || V_SEX_CD || ']');
DBMS_OUTPUT.PUT_LINE('V_STD_YEAR    : ' || '[' || V_STD_YEAR || ']');
DBMS_OUTPUT.PUT_LINE('V_PAY_AMT     : ' || '[' || V_PAY_AMT || ']');

IF V_SEX_CD = '1' THEN
INSERT INTO TB_EMP_PAY_BY_YEAR VALUES (V_EMP_NO, V_STD_YEAR, V_PAY_AMT);
END IF;
END LOOP;
CLOSE SELECT_TB_EMP;

COMMIT;
DBMS_OUTPUT.PUT_LINE('-----');
END SP_INSERT_TB_EMP_PAY_BY_YEAR;
/
```

- ❖ 입력한 년(YYYY) 기준 직원 별 연봉 액수를 TB_EMP_PAY_BY_YEAR 테이블에 저장함
- ❖ 성별이 남자인 직원에 한해서만 저장함

➤ 오라클 프로시저 실행

```
TRUNCATE TABLE TB_EMP_PAY_BY_YEAR;  
EXEC SP_INSERT_TB_EMP_PAY_BY_YEAR('2019');
```

- ❖ 입력 값을 2019로 함
- ❖ 2019년도 직원 별 급여 지급 합계를 구하게 됨

```
Microsoft Windows [Version 10.0.18363.900]  
(c) 2019 Microsoft Corporation. All rights reserved.  
  
C:\Users\dbmsexpert>SQLPLUS SQLD/1234  
  
SQL*Plus: Release 12.2.0.1.0 Production on Sat Jul 4 11:21:29 2020  
  
Copyright (c) 1982, 2016, Oracle. All rights reserved.  
  
Last Successful login time: Fri Jul 03 2020 22:23:22 +09:00  
  
Connected to:  
Oracle Database 12c Enterprise Edition Release 12.2.0.1.0 - 64bit Production  
  
SQL> EXEC SP_INSERT_TB_EMP_PAY_BY_YEAR('2019');  
  
PL/SQL procedure successfully completed.  
  
SQL>
```

❖ SQLPLUS에서 실행함

```
SELECT * FROM TB_EMP_PAY_BY_YEAR;
```

EM P_N O	STD_YEAR	PAY_AM T
1000000001	2019	46270000
1000000002	2019	53530000
1000000003	2019	45740000
1000000004	2019	49220000
1000000009	2019	48800000
1000000012	2019	48990000
1000000014	2019	58690000
1000000015	2019	49180000
1000000017	2019	49240000
1000000021	2019	46780000
1000000023	2019	46430000
1000000024	2019	47140000
1000000026	2019	50780000
1000000030	2019	50210000
1000000032	2019	53490000
1000000033	2019	50670000
1000000035	2019	49090000
1000000039	2019	51000000
9999999999	2019	44330000

❖ 데이터가 저장되어 있는 것을 확인

➤ 사용자 정의 함수란?

- ① 사용자 정의 함수는 프로시저처럼 SQL문을 IF/LOOP등의 LOGIC와 함께 데이터베이스에 저장해 놓은 명령문의 집합이다.
- ② SUM, AVG, NVL의 함수처럼 호출해서 사용할 수 있다.
- ③ 프로시저와 차이점은 반드시 한건을 되돌려 줘야 한다는 것이다.

➤ 사용자 정의 함수 생성

```
CREATE OR REPLACE FUNCTION
FUNC_EMP_CNT_BY_DEPT_CD(IN_DEPT_CD IN TB_DEPT.DEPT_CD%TYPE)
RETURN NUMBER IS V_EMP_CNT NUMBER;

BEGIN
SELECT COUNT(*) CNT
  INTO V_EMP_CNT
  FROM TB_EMP
 WHERE DEPT_CD = IN_DEPT_CD
;

RETURN V_EMP_CNT;
END;
/
```

❖ SQLPLUS에서 실행함

➤ 사용자 정의 함수 생성 - SQLPLUS

(c) 2019 Microsoft Corporation. All rights reserved.

C:\Users\dbmsexpert>SQLPLUS SQLD/1234

SQL*Plus: Release 12.2.0.1.0 Production on Sat Jul 4 11:29:13 2020

Copyright (c) 1982, 2016, Oracle. All rights reserved.

Last Successful login time: Sat Jul 04 2020 11:21:46 +09:00

Connected to:

Oracle Database 12c Enterprise Edition Release 12.2.0.1.0 - 64bit Production

```
SQL> CREATE OR REPLACE FUNCTION FUNC_EMP_CNT_BY_DEPT_CD(IN_DEPT_CD IN TB_DEPT.DEPT_CD%TYPE)
2 RETURN NUMBER IS V_EMP_CNT NUMBER;
3
4 BEGIN
5 SELECT COUNT(*) CNT
6 INTO V_EMP_CNT
7 FROM TB_EMP
8 WHERE DEPT_CD = IN_DEPT_CD
9 ;
10
11 RETURN V_EMP_CNT;
12 END;
13 /
```

Function created.

SQL>

➤ 사용자 정의 함수 호출

```
SELECT A.DEPT_CD
      , A.DEPT_NM
      , FUNC_EMP_CNT_BY_DEPT_CD(A.DEPT_CD) AS EMP_CNT
FROM TB_DEPT A
;
```

DEPT_CD	DEPT_NM	EM P_CNT
100001	운영본부	1
100002	지원팀	4
100003	기획팀	4
100004	디자인팀	4
100005	플랫폼사업본부	1
100006	데이터팀	4
100007	개발팀	4
100008	솔루션사업본부	1
100009	운영팀	4
100010	개발팀	4
100011	신사업본부	1
100012	인공지능팀	4
100013	빅데이터팀	4
999999	회장실	1

➤ 트리거란?

- ① 특정한 테이블에 INSERT, UPDATE, DELETE를 수행할 때 DBMS내에서 자동으로 동작하도록 작성된 프로그램이다.
- ② 즉 사용자가 직접 호출하는 것이 아니고 DBMS가 자동적으로 수행한다.
- ③ 트리거는 테이블과 뷰, DB작업을 대상으로 정의 할수 있으며 전체 트랜잭션 작업에 대해 발생하는 트리거와 각행에 대해 발생하는 트리거가 있다.

➤ 트리거 실습 준비

```
DROP TABLE TB_EMP_PAY_SUM;  
  
CREATE TABLE TB_EMP_PAY_SUM  
(  
    EMP_NO    CHAR(10)Not Null  
,    PAY_AMT_SUM NUMBER(15) Not Null  
,    CONSTRAINT PK_TB_SAL_HIS_SUM PRIMARY KEY(EMP_NO)  
)  
;  
  
INSERT INTO TB_EMP_PAY_SUM  
SELECT EMP_NO, SUM(PAY_AMT) FROM TB_SAL_HIS  
GROUP BY EMP_NO  
ORDER BY EMP_NO  
;  
  
COMMIT;
```

❖ 직원 별 급여지급 합계를 저장하는 테이블을 생성

➤ 트리거 생성

```
CREATE OR REPLACE TRIGGER TRIG_TB_SAL_HIS_INSERT
AFTER INSERT
ON TB_SAL_HIS
FOR EACH ROW
DECLARE
V_EMP_NO TB_SAL_HIS.EMP_NO%TYPE;

BEGIN
V_EMP_NO := :NEW.EMP_NO;

UPDATE TB_EMP_PAY_SUM A
SET A.PAY_AMT_SUM = A.PAY_AMT_SUM + :NEW.PAY_AMT
WHERE A.EMP_NO = V_EMP_NO;

IF SQL%NOTFOUND THEN
    INSERT INTO TB_EMP_PAY_SUM VALUES (V_EMP_NO, :NEW.PAY_AMT);
END IF;
END;
/
```

- ❖ SQLPLUS에서 실행함
- ❖ 해당 트리거는 TB_SAL_HIS 테이블에 INSERT시 INSERT하려는 EMP_NO가 TB_EMP_PAY_SUM 테이블에 존재하면 TB_EMP_PAY_SUM 테이블 내의 PAY_AMT_SUM 컬럼의 값을 UPDATE하고 존재하지 않는다면 TB_EMP_PAY_SUM 테이블에 새로운 행을 INSERT하는 역할을 한다.

➤ 트리거 생성 - SQLPLUS

SQL*Plus: Release 12.2.0.1.0 Production on Sat Jul 4 11:35:37 2020

Copyright (c) 1982, 2016, Oracle. All rights reserved.

Last Successful login time: Sat Jul 04 2020 11:35:02 +09:00

Connected to:

Oracle Database 12c Enterprise Edition Release 12.2.0.1.0 - 64bit Production

```
SQL> CREATE OR REPLACE TRIGGER TRIG_TB_SAL_HIS_INSERT
 2  AFTER INSERT
 3  ON TB_SAL_HIS
 4  FOR EACH ROW
 5  DECLARE
 6  V_EMP_NO TB_SAL_HIS.EMP_NO%TYPE;
 7
 8  BEGIN
 9  V_EMP_NO := :NEW.EMP_NO;
10
11  UPDATE TB_EMP_PAY_SUM A
12  SET A.PAY_AMT_SUM = A.PAY_AMT_SUM + :NEW.PAY_AMT
13  WHERE A.EMP_NO = V_EMP_NO;
14
15  IF SQL%NOTFOUND THEN
16      INSERT INTO TB_EMP_PAY_SUM VALUES (V_EMP_NO, :NEW.PAY_AMT);
17  END IF;
18  END;
19  /
```

Trigger created.

SQL>

➤ 트리거 실습

```
INSERT INTO TB_SAL_HIS VALUES (  
  ( SELECT TO_CHAR(NVL(MAX(SAL_HIS_NO), 0) + 1) AS SAL_HIS_NO FROM TB_SAL_HIS )  
  , TO_CHAR(SYSDATE, 'YYYYMMDD'), 1000000 , '9999999999'  
);  
COMMIT;
```

```
SELECT A.EMP_NO  
      , A.PAY_AMT_SUM  
FROM TB_EMP_PAY_SUM a  
WHERE EMP_NO = '9999999999'  
;
```

EMP_NO	PAY_AMT_SUM
9999999999	103110000

❖ TB_SAL_HIS 테이블에 데이터를 입력하면 TB_EMP_PAY_SUM 테이블에 데이터가 입력됨

➤ 트리거 실습 - 새로운 직원을 입력

```
ALTER TABLE TB_SAL_HIS DROP CONSTRAINT FK_TB_SAL_HIS01;
```

❖ 실습을 위해 존재하지 않는 직원번호를 입력하므로 참조 무결성 제약 조건 잠시 제거

```
INSERT INTO TB_SAL_HIS VALUES (  
  ( SELECT TO_CHAR(NVL(MAX(SAL_HIS_NO), 0) + 1) AS SAL_HIS_NO FROM TB_SAL_HIS )  
  , TO_CHAR(SYSDATE, 'YYYYMMDD'), 1000000 , '1234567890'  
);  
COMMIT;
```

```
SELECT A.EMP_NO  
      , A.PAY_AMT_SUM  
FROM TB_EMP_PAY_SUM A  
WHERE EMP_NO = '1234567890'  
;
```

EMP_NO	PAY_AMT_SUM
1234567890	1000000

```
DELETE FROM TB_SAL_HIS WHERE EMP_NO = '1234567890';  
COMMIT;
```

❖ 테스트 데이터를 삭제 후 참조 무결성 제약조건을 원상 복귀 시킴

```
ALTER TABLE SQLD.TB_SAL_HIS ADD CONSTRAINT FK_TB_SAL_HIS01 FOREIGN KEY (EMP_NO) REFERENCES SQLD.TB_EMP (EMP_NO) NOVALIDATE;
```

➤ 프로시저와 트리거의 차이점

프로시저	트리거
CREATE PROCEDURE 문법 사용	CREATE TRIGGER 문법 사용
EXECUTE/EXEC 명령어로 실행	생성 후 자동으로 실행
내부에서 COMMIT, ROLLBACK 실행가능	내부에서 COMMIT, ROLLBACK 실행 안됨

Chapter 04. SQL 활용

4-9. 연습문제

문제 1.

아래 SQL문은 한국데이터베이스진흥원과 오라클에서 발급한 자격증이 하나도 없는 직원의 직원 번호, 직원 명, 보유중인 자격증 코드, 자격증 정보를 출력하는 SQL문이다. ㉠, ㉡에 들어갈 연산자로 알맞는 것은?

```
SELECT A.EMP_NO, A.EMP_NM , B.CERTI_CD
, (SELECT L.CERTI_NM || '(' || L.ISSUE_INSTI_NM || ')' FROM TB_CERTI L WHERE L.CERTI_CD = B.CERTI_CD) AS CERTI_INFO
FROM TB_EMP A, TB_EMP_CERTI B
WHERE ㉠ (SELECT 1
        FROM TB_CERTI J
        , TB_EMP_CERTI K
        WHERE J.ISSUE_INSTI_NM IN ( '한국데이터베이스진흥원' , '오라클' )
        AND J.CERTI_CD = K.CERTI_CD
        AND K.EMP_NO ㉡ A.EMP_NO )
AND A.EMP_NO = B.EMP_NO
ORDER BY A.EMP_NO
;
```

- ① ㉠ : NOT EXISTS ㉡ : <>
- ② ㉠ : EXISTS ㉡ : =
- ③ ㉠ : EXISTS ㉡ : <>
- ④ ㉠ : NOT EXISTS ㉡ : =

문제 1.

아래 SQL문은 한국데이터베이스진흥원과 오라클에서 발급한 자격증이 하나도 없는 직원의 직원 번호, 직원 명, 보유중인 자격증 코드, 자격증 정보를 출력하는 SQL문이다. ㉠, ㉡에 들어갈 연산자로 알맞는 것은?

```

SELECT A.EMP_NO, A.EMP_NM , B.CERTI_CD
, (SELECT L.CERTI_NM || '(' || L.ISSUE_INSTI_NM || ')' FROM TB_CERTI L WHERE L.CERTI_CD = B.CERTI_CD) AS CERTI_INFO
FROM TB_EMP A, TB_EMP_CERTI B
WHERE ㉠ (SELECT 1
        FROM TB_CERTI J
        , TB_EMP_CERTI K
        WHERE J.ISSUE_INSTI_NM IN ( '한국데이터베이스진흥원' , '오라클' )
        AND J.CERTI_CD = K.CERTI_CD
        AND K.EMP_NO ㉡ A.EMP_NO )
AND A.EMP_NO = B.EMP_NO
ORDER BY A.EMP_NO
;
    
```

- ① ㉠ : NOT EXISTS ㉡ : <>
- ② ㉠ : EXISTS ㉡ : =
- ③ ㉠ : EXISTS ㉡ : <>
- ④ ㉠ : NOT EXISTS ㉡ : =

문제 2. TB_CUST_ORD 테이블은 고객 기준 특정 일자의 매출액을 저장하는 테이블이다.
아래와 같이 테이블을 생성하고 데이터를 입력하였다.

```
DROP TABLE TB_CUST_ORD;  
CREATE TABLE TB_CUST_ORD  
(  
    CUST_NO CHAR(10), SALE_DE CHAR(8), SALE_AMT NUMBER(15)  
);
```

```
INSERT INTO TB_CUST_ORD VALUES ('1000000001', '20190701', 100000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000001', '20190702', 30000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000001', '20190702', 100000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000002', '20190701', 200000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000002', '20190701', 20000);  
COMMIT;
```

아래 SQL문의 결과로 올바른 것은 무엇인가?

```
SELECT  
    CUST_NO, SALE_DE, SALE_AMT  
    , SUM(SALE_AMT) OVER(PARTITION BY CUST_NO ORDER BY SALE_DE ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS AMT  
FROM TB_CUST_ORD;  
;
```

①

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190702	100000	130000
1000000001	20190702	30000	130000
1000000001	20190701	100000	230000
1000000002	20190701	20000	220000
1000000002	20190701	200000	220000

②

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190701	100000	100000
1000000001	20190702	30000	230000
1000000001	20190702	100000	230000
1000000002	20190701	200000	220000
1000000002	20190701	20000	220000

③

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190701	100000	100000
1000000001	20190702	30000	130000
1000000001	20190702	100000	230000
1000000002	20190701	200000	200000
1000000002	20190701	20000	220000

④

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190701	100000	230000
1000000001	20190702	30000	230000
1000000001	20190702	100000	230000
1000000002	20190701	200000	220000
1000000002	20190701	20000	220000

문제 2. TB_CUST_ORD 테이블은 고객 기준 특정 일자의 매출액을 저장하는 테이블이다.
아래와 같이 테이블을 생성하고 데이터를 입력하였다.

```
DROP TABLE TB_CUST_ORD;  
CREATE TABLE TB_CUST_ORD  
(  
  CUST_NO CHAR(10), SALE_DE CHAR(8), SALE_AMT NUMBER(15)  
);
```

```
INSERT INTO TB_CUST_ORD VALUES ('1000000001', '20190701', 100000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000001', '20190702', 30000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000001', '20190702', 100000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000002', '20190701', 200000);  
INSERT INTO TB_CUST_ORD VALUES ('1000000002', '20190701', 20000);  
COMMIT;
```

아래 SQL문의 결과로 올바른 것은 무엇인가?

```
SELECT  
  CUST_NO, SALE_DE, SALE_AMT  
  , SUM(SALE_AMT) OVER(PARTITION BY CUST_NO ORDER BY SALE_DE ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS AMT  
FROM TB_CUST_ORD;  
;
```

①

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190702	100000	130000
1000000001	20190702	30000	130000
1000000001	20190701	100000	230000
1000000002	20190701	20000	220000
1000000002	20190701	200000	220000

②

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190701	100000	100000
1000000001	20190702	30000	230000
1000000001	20190702	100000	230000
1000000002	20190701	200000	220000
1000000002	20190701	20000	220000

③

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190701	100000	100000
1000000001	20190702	30000	130000
1000000001	20190702	100000	230000
1000000002	20190701	200000	200000
1000000002	20190701	20000	220000

④

CUST_NO	SALE_DE	SALE_AMT	AMT
1000000001	20190701	100000	230000
1000000001	20190702	30000	230000
1000000001	20190702	100000	230000
1000000002	20190701	200000	220000
1000000002	20190701	20000	220000

문제 3. 아래의 SQL문에 대한 설명으로 가장 부적절한 것은?

```
SELECT SUM(NVL(B.PAY_AMT, 0)) AS PAY_AMT_SUM
FROM TB_EMP A, TB_SAL_HIS B
WHERE A.EMP_NM = '김회장';
```

- ① 조인 조건이 없다고 해서 문법상 오류가 일어나지는 않는다.
- ② SUM(NVL(B.PAY_AMT, 0)) 연산에 비효율이 존재한다.
- ③ 조인 조건이 없어서 CARTESIAN PRODUCT 연산이 발생한다.
- ④ 조인 조건이 없어서 결과 건수가 여러 건이 된다.

문제 4. 아래 SQL문은 생년월일 순으로 오름차순의 순위를 구하는 SQL문이다. SQL문의 결과로 봤을 때 ㉠에 들어갈 분석 함수는 무엇인가?

```
SELECT EMP_NO, EMP_NM, BIRTH_DE, ㉠() OVER(ORDER BY BIRTH_DE) BIRTH_DE_RANK
FROM TB_EMP
WHERE SEX_CD = '1'
AND BIRTH_DE >= '19800000';
```

- ① DENSE_RANK
- ② ROW_NUMBER
- ③ RANK
- ④ ROWNUM

EMP_NO	EMP_NM	BIRTH_DE	BIRTH_DE_RANK
1000000001	이경오	19840612	1
1000000024	박선영	19870615	2
1000000033	홍사기	19870615	2
1000000004	이승준	19870623	4
1000000017	이겸손	19880124	5
1000000009	박태범	19880629	6
1000000002	이현승	19880705	7
1000000012	김호형	19910227	8
1000000021	김열호	19910227	8
1000000003	이정수	19911224	10
1000000039	이박력	19940709	11
1000000030	이규호	19970627	12
1000000015	이정직	19980715	13

문제 3. 아래의 SQL문에 대한 설명으로 가장 부적절한 것은?

```
SELECT SUM(NVL(B.PAY_AMT, 0)) AS PAY_AMT_SUM
FROM TB_EMP A, TB_SAL_HIS B
WHERE A.EMP_NM = '김회장';
```

- ① 조인 조건이 없다고 해서 문법상 오류가 일어나지는 않는다.
- ② SUM(NVL(B.PAY_AMT, 0)) 연산에 비효율이 존재한다.
- ③ 조인 조건이 없어서 CARTESIAN PRODUCT 연산이 발생한다.
- ④ 조인 조건이 없어서 결과 건수가 여러 건이 된다.

문제 4. 아래 SQL문은 생년월일 순으로 오름차순의 순위를 구하는 SQL문이다. SQL문의 결과로 봤을 때 ㉠에 들어갈 분석 함수는 무엇인가?

```
SELECT EMP_NO, EMP_NM, BIRTH_DE, ㉠() OVER(ORDER BY BIRTH_DE) BIRTH_DE_RANK
FROM TB_EMP
WHERE SEX_CD = '1'
AND BIRTH_DE >= '19800000';
```

- ① DENSE_RANK
- ② ROW_NUMBER
- ③ RANK
- ④ ROWNUM

EMP_NO	EMP_NM	BIRTH_DE	BIRTH_DE_RANK
1000000001	이경오	19840612	1
1000000024	박선영	19870615	2
1000000033	홍사기	19870615	2
1000000004	이승준	19870623	4
1000000017	이겸손	19880124	5
1000000009	박태범	19880629	6
1000000002	이현승	19880705	7
1000000012	김호형	19910227	8
1000000021	김열호	19910227	8
1000000003	이정수	19911224	10
1000000039	이박력	19940709	11
1000000030	이규호	19970627	12
1000000015	이정직	19980715	13

문제 5. 다음 중 아래의 SQL문의 결과와 동일한 SQL문을 작성하고자 한다.

```
SELECT A.EMP_NO, A.EMP_NM, NVL(A.DIRECT_MANAGER_EMP_NO, '관리자없음') AS 직속관리자사원번호
FROM TB_EMP A
WHERE A.EMP_NM = '김회장'
;
```

㉠에 들어갈 알맞은 SQL문은 무엇인가?

```
SELECT A.EMP_NO, A.EMP_NM
, CASE WHEN (㉠) THEN '관리자없음' END AS 직속관리자사원번호
FROM TB_EMP A
WHERE A.EMP_NM = '김회장'
;
```

- ① A.DIRECT_MANAGER_EMP_NO IS NOT NULL
- ② A.DIRECT_MANAGER_EMP_NO = 'NULL'
- ③ A.DIRECT_MANAGER_EMP_NO = NULL
- ④ A.DIRECT_MANAGER_EMP_NO IS NULL

문제 5. 다음 중 아래의 SQL문의 결과와 동일한 SQL문을 작성하고자 한다.

```
SELECT A.EMP_NO, A.EMP_NM, NVL(A.DIRECT_MANAGER_EMP_NO, '관리자없음') AS 직속관리자사원번호
FROM TB_EMP A
WHERE A.EMP_NM = '김회장'
;
```

㉠에 들어갈 알맞은 SQL문은 무엇인가?

```
SELECT A.EMP_NO, A.EMP_NM
, CASE WHEN (㉠) THEN '관리자없음' END AS 직속관리자사원번호
FROM TB_EMP A
WHERE A.EMP_NM = '김회장'
;
```

- ① A.DIRECT_MANAGER_EMP_NO IS NOT NULL
- ② A.DIRECT_MANAGER_EMP_NO = 'NULL'
- ③ A.DIRECT_MANAGER_EMP_NO = NULL
- ④ A.DIRECT_MANAGER_EMP_NO IS NULL

문제 6. 아래와 같은 테이블을 생성 후 데이터를 입력하였다.
입력한 데이터를 참고하여 가장 올바른 것은 무엇인가?

```
CREATE TABLE TB_COL_SUM_TEST
(
  COL_1 NUMBER(15)
, COL_2 NUMBER(15)
, COL_3 NUMBER(15)
)
;

INSERT INTO TB_COL_SUM_TEST VALUES(10, 20, NULL);
INSERT INTO TB_COL_SUM_TEST VALUES(15, NULL, NULL);
INSERT INTO TB_COL_SUM_TEST VALUES(50, 70, 20);
COMMIT;

SELECT COL_1, COL_2, COL_3 FROM TB_COL_SUM_TEST;
```

COL_1	COL_2	COL_3
10	20	(NULL)
15	(NULL)	(NULL)
50	70	20

- ① SELECT SUM(COL_2) FROM TB_COL_SUM_TEST; 의 결과는 NULL이다.
- ② SELECT SUM(COL_1 + COL_2 + COL_3) FROM TB_COL_SUM_TEST; 의 결과는 185이다.
- ③ SELECT SUM(COL_2 + COL_3) FROM TB_COL_SUM_TEST; 의 결과는 110이다.
- ④ SELECT SUM(COL_2) + SUM(COL_3) FROM TB_COL_SUM_TEST; 의 결과는 110이다.

문제 6. 아래와 같은 테이블을 생성 후 데이터를 입력하였다.
입력한 데이터를 참고하여 가장 올바른 것은 무엇인가?

```
CREATE TABLE TB_COL_SUM_TEST
(
  COL_1 NUMBER(15)
, COL_2 NUMBER(15)
, COL_3 NUMBER(15)
)
;

INSERT INTO TB_COL_SUM_TEST VALUES(10, 20, NULL);
INSERT INTO TB_COL_SUM_TEST VALUES(15, NULL, NULL);
INSERT INTO TB_COL_SUM_TEST VALUES(50, 70, 20);
COMMIT;

SELECT COL_1, COL_2, COL_3 FROM TB_COL_SUM_TEST;
```

COL_1	COL_2	COL_3
10	20	(NULL)
15	(NULL)	(NULL)
50	70	20

- ① SELECT SUM(COL_2) FROM TB_COL_SUM_TEST; 의 결과는 NULL이다.
- ② SELECT SUM(COL_1 + COL_2 + COL_3) FROM TB_COL_SUM_TEST; 의 결과는 185이다.
- ③ SELECT SUM(COL_2 + COL_3) FROM TB_COL_SUM_TEST; 의 결과는 110이다.
- ④ SELECT SUM(COL_2) + SUM(COL_3) FROM TB_COL_SUM_TEST; 의 결과는 110이다.

문제 7. 아래와 같은 테이블이 생성되고 데이터가 입력되었다. SQL문의 결과로 옳은 것은?

```
CREATE TABLE TB_ORD_TEST
(
  ORD_NO CHAR(10)
, ORD_DE CHAR(8)
, ORD_AMT NUMBER
, ADD_AMT NUMBER
, ORD_MM CHAR(6)
);

INSERT INTO TB_ORD_TEST VALUES('0000000001', '20200101', 500, 50, '202001');
COMMIT;
```

```
SELECT * FROM TB_ORD_TEST;
```

ORD_NO	ORD_DE	ORD_AMT	ADD_AMT	ORD_MM
0000000001	20200101	500	50	202001

```
SELECT COUNT(*) CNT, NVL(SUM(ORD_AMT), 0) AS ORD_AMT_SUM
FROM TB_ORD_TEST
WHERE ORD_MM = '202010'
GROUP BY ORD_MM
;
```

- ① 0, 0
- ② 0, NULL
- ③ 1, 500
- ④ 공집합

문제 7. 아래와 같은 테이블이 생성되고 데이터가 입력되었다. SQL문의 결과로 옳은 것은?

```
CREATE TABLE TB_ORD_TEST
(
  ORD_NO CHAR(10)
, ORD_DE CHAR(8)
, ORD_AMT NUMBER
, ADD_AMT NUMBER
, ORD_MM CHAR(6)
);

INSERT INTO TB_ORD_TEST VALUES('0000000001', '20200101', 500, 50, '202001');
COMMIT;
```

```
SELECT * FROM TB_ORD_TEST;
```

ORD_NO	ORD_DE	ORD_AMT	ADD_AMT	ORD_MM
0000000001	20200101	500	50	202001

```
SELECT COUNT(*) CNT, NVL(SUM(ORD_AMT), 0) AS ORD_AMT_SUM
FROM TB_ORD_TEST
WHERE ORD_MM = '202010'
GROUP BY ORD_MM
;
```

- ① 0, 0
- ② 0, NULL
- ③ 1, 500
- ④ 공집합

문제 8. 아래의 SQL문에 대한 설명으로 가장 적절한 것은?

```
SELECT *  
FROM TB_EMP A, TB_DEPT B  
WHERE A.DEPT_CD = '100001'  
AND B.DEPT_CD = '100001'  
;
```

```
SELECT *  
FROM TB_EMP A, TB_DEPT B  
WHERE A.DEPT_CD = '100001'  
AND A.DEPT_CD = B.DEPT_CD  
;
```

- ① 두번째 SQL문은 온라인 프로그램에서 성능 상 유리하다.
- ② 2개의 SQL문은 성능상 확연한 차이가 난다.
- ③ 두 SQL문의 결과는 동일하다.
- ④ 두 SQL문의 결과는 다르다.

문제 8. 아래의 SQL문에 대한 설명으로 가장 적절한 것은?

```
SELECT *  
FROM TB_EMP A, TB_DEPT B  
WHERE A.DEPT_CD = '100001'  
AND B.DEPT_CD = '100001'  
;
```

```
SELECT *  
FROM TB_EMP A, TB_DEPT B  
WHERE A.DEPT_CD = '100001'  
AND A.DEPT_CD = B.DEPT_CD  
;
```

- ① 두번째 SQL문은 온라인 프로그램에서 성능 상 유리하다.
- ② 2개의 SQL문은 성능상 확연한 차이가 난다.
- ③ 두 SQL문의 결과는 동일하다.
- ④ 두 SQL문의 결과는 다르다.

문제 9. 다음 중 절차 형 SQL모듈에 대한 설명으로 가장 부적절한 것은?

- ① 사용자 정의 함수는 절차 형 SQL을 로직과 함께 데이터베이스내에 저장해 놓은 명령문의 집합을 의미한다.
- ② 트리거는 특정한 테이블에 DML문이 수행되었을 경우 DBMS내에서 자동으로 작동한다.
- ③ 스토어드 프로시저는 절차 형 SQL문을 로직과 함께 DBMS내에서 저장해 놓은 명령문의 집합이다.
- ④ 사용자 정의 함수를 사용하면 모듈화 처리가 이루어지므로 성능상 유리하다.

문제 10. 계층 형 쿼리에 대한 설명으로 옳바르지 않는 것은?

- ① START WITH절은 계층 구조의 시작점을 지정하는 구문이다.
- ② 루트 노드의 LEVEL값은 0부터 시작한다.
- ③ 순방향 전개 란 부모노드로부터 자식 노드 방향으로 전개하는 것을 말한다.
- ④ ORDER SIBLINGS BY는 형제 노드 사이에서 정렬을 지정하는 구문이다.

문제 11. 다음 중 서브 쿼리에 대한 설명으로 가장 부적절한 것은?

- ① 서브 쿼리의 특성 상 메인 쿼리는 주문 테이블(M)이고 서브 쿼리는 사원 테이블(1)일 경우 질의 결과는 M레벨인 주문 테이블 레벨로 나온다.
- ② 서브 쿼리의 특성 상 메인 쿼리는 주문 테이블(M)이고 서브 쿼리는 사원 테이블(1)일 경우 질의 결과는 1레벨인 사원 테이블 레벨로 나온다.
- ③ 메인쿼리에서 서브 쿼리의 컬럼을 사용할 수 없다.
- ④ 서브쿼리에서 메인 쿼리의 컬럼은 사용할 수 있다.

문제 9. 다음 중 절차 형 SQL모듈에 대한 설명으로 가장 부적절한 것은?

- ① 사용자 정의 함수는 절차 형 SQL을 로직과 함께 데이터베이스내에 저장해 놓은 명령문의 집합을 의미한다.
- ② 트리거는 특정한 테이블에 DML문이 수행되었을 경우 DBMS내에서 자동으로 작동한다.
- ③ 스토어드 프로시저는 절차 형 SQL문을 로직과 함께 DBMS내에서 저장해 놓은 명령문의 집합이다.
- ④ 사용자 정의 함수를 사용하면 모듈화 처리가 이루어지므로 성능상 유리하다.

문제 10. 계층 형 쿼리에 대한 설명으로 옳바르지 않는 것은?

- ① START WITH절은 계층 구조의 시작점을 지정하는 구문이다.
- ② 루트 노드의 LEVEL값은 0부터 시작한다.
- ③ 순방향 전개 란 부모노드로부터 자식 노드 방향으로 전개하는 것을 말한다.
- ④ ORDER SIBLINGS BY는 형제 노드 사이에서 정렬을 지정하는 구문이다.

문제 11. 다음 중 서브 쿼리에 대한 설명으로 가장 부적절한 것은?

- ① 서브 쿼리의 특성 상 메인 쿼리는 주문 테이블(M)이고 서브 쿼리는 사원 테이블(1)일 경우 질의 결과는 M레벨인 주문 테이블 레벨로 나온다.
- ② 서브 쿼리의 특성 상 메인 쿼리는 주문 테이블(M)이고 서브 쿼리는 사원 테이블(1)일 경우 질의 결과는 1레벨인 사원 테이블 레벨로 나온다.
- ③ 메인쿼리에서 서브 쿼리의 컬럼을 사용할 수 없다.
- ④ 서브쿼리에서 메인 쿼리의 컬럼은 사용할 수 있다.

문제 12. 스칼라 서브 쿼리에 대한 설명으로 가장 적절한 것은?

- ① 내부적으로 PK에 의한 UNIQUE INDEX SCAN작업을 한다.
- ② 하나의 로우에 해당하는 결과 건수는 1건 이상이어야 한다.
- ③ MIN또는 MAX함수를 이용해야 한다.
- ④ 결과 컬럼의 개수는 1개여야 한다.

문제 13. 일반적으로 FROM절에 정의된 후 먼저 수행되어 SQL문 내에서 절차 성을 주는 효과를 볼 수 있는 것은 어떤 유형의 서브쿼리인가?

- ① 스칼라서브쿼리
- ② 연관 서브 쿼리
- ③ 비 연관 서브 쿼리
- ④ 인라인 뷰 서브 쿼리

문제 14. 다음 중 서브 쿼리에 대한 설명 중 틀린 것은?

- ① 다중 행 연산자는 IN, ANY, ALL이 있으며 서브 쿼리의 결과로 하나 이상의 데이터가 리턴 되는 서브쿼리이다.
- ② TOP-N 서브 쿼리는 인라인 뷰의 정렬된 데이터를 ROWNUM을 이용해 결과 행수를 제한하는 쿼리이다.
- ③ 인라인 뷰는 FROM절에 위치하며 실질적인 객체는 아니지만 SQL문 내에서 마치 뷰나 테이블과 같은 역할을 한다.
- ④ 인라인 뷰 서브 쿼리는 집합을 효율적으로 관리하므로 SQL문 전체의 처리속도를 빠르게 한다.

문제 12. 스칼라 서브 쿼리에 대한 설명으로 가장 적절한 것은?

- ① 내부적으로 PK에 의한 UNIQUE INDEX SCAN작업을 한다.
- ② 하나의 로우에 해당하는 결과 건수는 1건 이상이어야 한다.
- ③ MIN또는 MAX함수를 이용해야 한다.
- ④ 결과 컬럼의 개수는 1개여야 한다.

문제 13. 일반적으로 FROM절에 정의된 후 먼저 수행되어 SQL문 내에서 절차 성을 주는 효과를 볼 수 있는 것은 어떤 유형의 서브쿼리인가?

- ① 스칼라서브쿼리
- ② 연관 서브 쿼리
- ③ 비 연관 서브 쿼리
- ④ 인라인 뷰 서브 쿼리

문제 14. 다음 중 서브 쿼리에 대한 설명 중 틀린 것은?

- ① 다중 행 연산자는 IN, ANY, ALL이 있으며 서브 쿼리의 결과로 하나 이상의 데이터가 리턴 되는 서브쿼리이다.
- ② TOP-N 서브 쿼리는 인라인 뷰의 정렬된 데이터를 ROWNUM을 이용해 결과 행수를 제한하는 쿼리이다.
- ③ 인라인 뷰는 FROM절에 위치하며 실질적인 객체는 아니지만 SQL문 내에서 마치 뷰나 테이블과 같은 역할을 한다.
- ④ 인라인 뷰 서브 쿼리는 집합을 효율적으로 관리하므로 SQL문 전체의 처리속도를 빠르게 한다.

문제 15. 다음 중 소계, 증계, 합계와 같은 데이터 집계에 적합한 GROUP 함수 2가지는 무엇인가?

- ① ROLLUP, SUM
- ② ROLLUP, CUBE
- ③ ROLLUP, GROUPING
- ④ CUBE, SUM

문제 16. 그룹 내 순위 관련 윈도우 함수의 특징이 아닌 것은?

- ① RANK는 동일한 값에 대해서 동일한 순위를 부여한다.
- ② DENSE_RANK 함수는 RANK 함수와 유사하지만 동일한 순위를 하나의 건수로 취급하는 것이 차이점이다.
- ③ ROW_NUMBER 함수는 동일한 값에 대해서 동일 순위를 부여하지만 DENSE_RANK 처럼 하나의 건수로 취급하진 않는다.
- ④ ROW_NUMBER 함수는 동일한 값에 대해서도 다른 순위를 부여한다.

문제 17. 다음 중 절차 형 SQL을 이용하여 주로 만드는 것이 아닌 것은?

- ① 프로시저
- ② 트리거
- ③ 사용자정의 함수
- ④ 내장 함수

문제 15. 다음 중 소계, 증계, 합계와 같은 데이터 집계에 적합한 GROUP 함수 2가지는 무엇인가?

- ① ROLLUP, SUM
- ② **ROLLUP, CUBE**
- ③ ROLLUP, GROUPING
- ④ CUBE, SUM

문제 16. 그룹 내 순위 관련 윈도우 함수의 특징이 아닌 것은?

- ① RANK는 동일한 값에 대해서 동일한 순위를 부여한다.
- ② DENSE_RANK 함수는 RANK 함수와 유사하지만 동일한 순위를 하나의 건수로 취급하는 것이 차이점이다.
- ③ **ROW_NUMBER 함수는 동일한 값에 대해서 동일 순위를 부여하지만 DENSE_RANK 처럼 하나의 건수로 취급 하진 않는다.**
- ④ ROW_NUMBER 함수는 동일한 값에 대해서도 다른 순위를 부여한다.

문제 17. 다음 중 절차 형 SQL을 이용하여 주로 만드는 것이 아닌 것은?

- ① 프로시저
- ② 트리거
- ③ 사용자정의함수
- ④ **내장 함수**

문제 18. 아래와 같은 테이블이 생성되고 데이터가 입력되었다.

```
CREATE TABLE TB_COL_SUM_TEST2
(
  COL_1 NUMBER(15)
, COL_2 NUMBER(15)
, COL_3 NUMBER(15)
, COL_4 NUMBER(15)
)
;

INSERT INTO TB_COL_SUM_TEST2 VALUES(NULL, NULL, 10, 20);
INSERT INTO TB_COL_SUM_TEST2 VALUES(10, 15, 20, 40);
INSERT INTO TB_COL_SUM_TEST2 VALUES(NULL, 10, NULL, NULL);
COMMIT;
```

SELECT * FROM TB_COL_SUM_TEST2;

COL_1	COL_2	COL_3	COL_4
-----	-----	-----	-----
NULL	NULL	10	20
10	15	20	40
NULL	10	NULL	NULL

SQL문의 결과로 옳은 것은?

```
SELECT SUM(COL_1) + SUM(COL_2+COL_3) + AVG(COL_4)
FROM TB_COL_SUM_TEST2;
```

- ① NULL
- ② 65
- ③ 75
- ④ 95

문제 18. 아래와 같은 테이블이 생성되고 데이터가 입력되었다.

```
CREATE TABLE TB_COL_SUM_TEST2
(
  COL_1 NUMBER(15)
, COL_2 NUMBER(15)
, COL_3 NUMBER(15)
, COL_4 NUMBER(15)
)
;

INSERT INTO TB_COL_SUM_TEST2 VALUES(NULL, NULL, 10, 20);
INSERT INTO TB_COL_SUM_TEST2 VALUES(10, 15, 20, 40);
INSERT INTO TB_COL_SUM_TEST2 VALUES(NULL, 10, NULL, NULL);
COMMIT;
```

SELECT * FROM TB_COL_SUM_TEST2;

COL_1	COL_2	COL_3	COL_4
-----	-----	-----	-----
NULL	NULL	10	20
10	15	20	40
NULL	10	NULL	NULL

SQL문의 결과로 옳은 것은?

```
SELECT SUM(COL_1) + SUM(COL_2+COL_3) + AVG(COL_4)
FROM TB_COL_SUM_TEST2;
```

- ① NULL
- ② 65
- ③ 75
- ④ 95

문제 19. 아래의 테이블에는 2020년 6월 29일의 코스피, 코스닥의 지수 데이터가 저장되어 있다.

```

DROP TABLE TB_REAL_IDX PURGE;

CREATE TABLE TB_REAL_IDX
(
    SEQ NUMBER(15)
    , SECTOR_NM VARCHAR2(50)
    , STD_DE CHAR(8)
    , STD_TM CHAR(6)
    , CUR_IDX NUMBER(15, 2)
    , CONSTRAINT PK_TB_REAL_IDX PRIMARY KEY(SEQ)
)
;

```

```

INSERT INTO TB_REAL_IDX
SELECT ROWNUM AS RNUM
    , '코스피' AS SECTOR_NM
    , '20200629' AS STD_DE
    , TO_CHAR(TO_DATE('090000', 'HH24MISS') + 1/24/60/60*(ROWNUM*60), 'HH24MISS') AS HH24MISS
    , CUR_IDX
FROM
(
    SELECT
        ROUND(DBMS_RANDOM.VALUE(2000.00, 2050.99), 2) AS CUR_IDX
    FROM DUAL CONNECT BY LEVEL <= 390
    ORDER BY CUR_IDX
)
UNION ALL
SELECT ROWNUM+390 AS RNUM
    , '코스닥' AS SECTOR_NM
    , '20200629' AS STD_DE
    , TO_CHAR(TO_DATE('090000', 'HH24MISS') + 1/24/60/60*(ROWNUM*60), 'HH24MISS') AS HH24MISS
    , CUR_IDX
FROM
(
    SELECT
        ROUND(DBMS_RANDOM.VALUE(700.00, 725.99), 2) AS CUR_IDX
    FROM DUAL CONNECT BY LEVEL <= 390
    ORDER BY CUR_IDX
)
;

COMMIT;

```

문제 19. 코스피, 코스닥 별 최고가를 찍은 시간 최저가를 찍은 시간을 구하는 SQL문을 작성하시오.

문제 19. 코스피, 코스닥 별 최고가를 찍은 시간 최저가를 찍은 시간을 구하는 SQL문을 작성하시오.

```
SELECT A.STD_DE AS "기준일자"
      , A.SECTOR_NM AS "지수명"
      , A.CUR_IDX_MAX AS "최고지수"
      , B.STD_TM AS "최고지수시점"
      , A.CUR_IDX_MIN AS "최저지수"
      , C.STD_TM AS "최저지수시점"
FROM
(
  SELECT A.STD_DE
        , A.SECTOR_NM
        , MAX(CUR_IDX) CUR_IDX_MAX
        , MIN(CUR_IDX) CUR_IDX_MIN
  FROM TB_REAL_IDX A
  WHERE STD_DE = '20200629'
  GROUP BY A.STD_DE, A.SECTOR_NM
) A
, TB_REAL_IDX B
, TB_REAL_IDX C
WHERE A.STD_DE = B.STD_DE
AND A.SECTOR_NM = B.SECTOR_NM
AND A.CUR_IDX_MAX = B.CUR_IDX
AND A.STD_DE = C.STD_DE
AND A.SECTOR_NM = C.SECTOR_NM
AND A.CUR_IDX_MIN = C.CUR_IDX
;
```

기준일자	지수명	최고지수	최고지수시점	최저지수	최저지수시점
20200629	코스피	2050.87	153000	2000.03	90100
20200629	코스닥	725.89	153000	700.02	90100

```
SELECT A.STD_DE AS "기준일자"
      , A.SECTOR_NM AS "지수명"
      , MAX(CUR_IDX) AS "최고지수"
      , MAX(STD_TM) KEEP(DENSE_RANK LAST ORDER BY A.CUR_IDX) AS "최고지수시점"
      , MIN(CUR_IDX) AS "최저지수"
      , MAX(STD_TM) KEEP(DENSE_RANK FIRST ORDER BY A.CUR_IDX) AS "최저지수시점"
FROM TB_REAL_IDX A
WHERE STD_DE = '20200629'
GROUP BY A.STD_DE, A.SECTOR_NM
;
```

기준일자	지수명	최고지수	최고지수시점	최저지수	최저지수시점
20200629	코스닥	725.89	153000	700.02	90100
20200629	코스피	2050.87	153000	2000.03	90100

문제 20. 다음과 같은 2개의 테이블에 데이터가 저장되어 있다.

```
DROP TABLE TB_MAIN;
DROP TABLE TB_SUB;
CREATE TABLE TB_MAIN
(
    SEQ NUMBER
    , VAL CHAR(1)
);
CREATE TABLE TB_SUB
(
    SEQ NUMBER
    , VAL CHAR(1)
);
INSERT INTO TB_MAIN VALUES ( 1, 'A' );
INSERT INTO TB_MAIN VALUES ( 2, NULL);
INSERT INTO TB_MAIN VALUES ( 3, 'B' );
INSERT INTO TB_MAIN VALUES ( 4, 'C' );

INSERT INTO TB_SUB VALUES ( 1, 'A' );
INSERT INTO TB_SUB VALUES ( 2, NULL);
INSERT INTO TB_SUB VALUES ( 3, 'B' );
COMMIT;
```

```
SELECT * FROM TB_MAIN;
```

```
SELECT * FROM TB_SUB;
```

SEQ	VAL
1	A
2	NULL
3	B
4	C

SEQ	VAL
1	A
2	NULL
3	B

각 SQL 문의 결과값이 잘못된 것은 무엇인가?

① `SELECT * FROM TB_MAIN A WHERE A.VAL IN (SELECT B.VAL FROM TB_SUB B);`

SEQ	VAL
1	A
3	B

② `SELECT * FROM TB_MAIN A WHERE A.VAL NOT IN (SELECT B.VAL FROM TB_SUB B);`

SEQ	VAL
4	C

③ `SELECT * FROM TB_MAIN A WHERE EXISTS
 (SELECT 1 FROM TB_SUB B WHERE B.VAL = A.VAL);`

SEQ	VAL
1	A
3	B

④ `SELECT * FROM TB_MAIN A WHERE NOT EXISTS
 (SELECT 1 FROM TB_SUB B WHERE B.VAL = A.VAL);`

SEQ	VAL
4	C
2	

문제 20. 다음과 같은 2개의 테이블에 데이터가 저장되어 있다.

```
DROP TABLE TB_MAIN;  
DROP TABLE TB_SUB;  
CREATE TABLE TB_MAIN  
(  
    SEQ NUMBER  
    , VAL CHAR(1)  
);  
CREATE TABLE TB_SUB  
(  
    SEQ NUMBER  
    , VAL CHAR(1)  
);  
INSERT INTO TB_MAIN VALUES ( 1, 'A' );  
INSERT INTO TB_MAIN VALUES ( 2, NULL);  
INSERT INTO TB_MAIN VALUES ( 3, 'B' );  
INSERT INTO TB_MAIN VALUES ( 4, 'C' );  
  
INSERT INTO TB_SUB VALUES ( 1, 'A' );  
INSERT INTO TB_SUB VALUES ( 2, NULL);  
INSERT INTO TB_SUB VALUES ( 3, 'B' );  
COMMIT;
```

```
SELECT * FROM TB_MAIN;
```

```
SELECT * FROM TB_SUB;
```

SEQ	VAL	SEQ	VAL
1	A	1	A
2		2	
3	B	3	B
4	C		

각 SQL 문의 결과값이 잘못된 것은 무엇인가?

① `SELECT * FROM TB_MAIN A WHERE A.VAL IN (SELECT B.VAL FROM TB_SUB B);`

SEQ	VAL
1	A
3	B

② `SELECT * FROM TB_MAIN A WHERE A.VAL NOT IN (SELECT B.VAL FROM TB_SUB B);`

SEQ	VAL
4	C

③ `SELECT * FROM TB_MAIN A WHERE EXISTS
 (SELECT 1 FROM TB_SUB B WHERE B.VAL = A.VAL);`

SEQ	VAL
1	A
3	B

④ `SELECT * FROM TB_MAIN A WHERE NOT EXISTS
 (SELECT 1 FROM TB_SUB B WHERE B.VAL = A.VAL);`

SEQ	VAL
4	C
2	

문제 21. 뷰의 특징과 그에 대한 설명이다. 잘못된 것은 무엇인가?

- ① 독립성 : 테이블 구조가 변경되어도 뷰를 사용하는 응용프로그램은 변경하지 않아도 된다.
- ② 편리성 : 복잡한 질의를 뷰로 생성함으로써 관련 질의를 단순하게 작성할 수 있다.
- ③ 물리성 : 뷰는 DBMS내부의 객체로 존재하므로 해당 데이터에 대한 물리적인 관리를 할 수 있다.
- ④ 보안성 : 뷰를 생성할 때 민감한 정보를 제외하고 생성함으로써 보안 성을 높일 수 있다.

문제 21. 뷰의 특징과 그에 대한 설명이다. 잘못된 것은 무엇인가?

- ① 독립성 : 테이블 구조가 변경되어도 뷰를 사용하는 응용프로그램은 변경하지 않아도 된다.
- ② 편리성 : 복잡한 질의를 뷰로 생성함으로써 관련 질의를 단순하게 작성할 수 있다.
- ③ 물리성 : 뷰는 DBMS내부의 객체로 존재하므로 해당 데이터에 대한 물리적인 관리를 할 수 있다.
- ④ 보안성 : 뷰를 생성할 때 민감한 정보를 제외하고 생성함으로써 보안 성을 높일 수 있다.

문제 22. 아래와 같은 두개의 테이블이 있다. 두개의 테이블을 조인 시 결과 건수를 차례대로 올바르게 기재한 것은?

```
CREATE TABLE TB_JOIN_1
(
    KEY1 CHAR(1)
    , VAL1 CHAR(3)
    , VAL2 CHAR(3)
);

CREATE TABLE TB_JOIN_2
(
    KEY2 CHAR(1)
    , VAL3 CHAR(3)
    , VAL4 CHAR(3)
);

INSERT INTO TB_JOIN_1 VALUES ('A', '111', '222');
INSERT INTO TB_JOIN_1 VALUES ('B', '222', '333');
INSERT INTO TB_JOIN_1 VALUES ('C', '333', '444');
INSERT INTO TB_JOIN_1 VALUES ('D', '444', '555');

INSERT INTO TB_JOIN_2 VALUES ('D', '555', '666');
INSERT INTO TB_JOIN_2 VALUES ('E', '666', '777');
INSERT INTO TB_JOIN_2 VALUES ('F', '777', '888');

COMMIT;

SELECT COUNT(*) FROM TB_JOIN_1 A INNER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A LEFT OUTER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A RIGHT OUTER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A FULL OUTER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A CROSS JOIN TB_JOIN_2 B ;
```

- ① 1, 4, 3, 5, 12
- ② 1, 4, 4, 6, 12
- ③ 1, 4, 3, 6, 12
- ④ 1, 3, 3, 6, 12

문제 22. 아래와 같은 두개의 테이블이 있다. 두개의 테이블을 조인 시 결과 건수를 차례대로 올바르게 기재한 것은?

```
CREATE TABLE TB_JOIN_1
(
    KEY1 CHAR(1)
    , VAL1 CHAR(3)
    , VAL2 CHAR(3)
);

CREATE TABLE TB_JOIN_2
(
    KEY2 CHAR(1)
    , VAL3 CHAR(3)
    , VAL4 CHAR(3)
);

INSERT INTO TB_JOIN_1 VALUES ('A', '111', '222');
INSERT INTO TB_JOIN_1 VALUES ('B', '222', '333');
INSERT INTO TB_JOIN_1 VALUES ('C', '333', '444');
INSERT INTO TB_JOIN_1 VALUES ('D', '444', '555');

INSERT INTO TB_JOIN_2 VALUES ('D', '555', '666');
INSERT INTO TB_JOIN_2 VALUES ('E', '666', '777');
INSERT INTO TB_JOIN_2 VALUES ('F', '777', '888');

COMMIT;

SELECT COUNT(*) FROM TB_JOIN_1 A INNER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A LEFT OUTER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A RIGHT OUTER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A FULL OUTER JOIN TB_JOIN_2 B ON (A.KEY1 = B.KEY2);
SELECT COUNT(*) FROM TB_JOIN_1 A CROSS JOIN TB_JOIN_2 B ;
```

- ① 1, 4, 3, 5, 12
- ② 1, 4, 4, 6, 12
- ③ 1, 4, 3, 6, 12
- ④ 1, 3, 3, 6, 12

문제 23. 아래와 같이 테이블을 생성하고 데이터를 입력하였다. 문제의 SQL문과 동일한 결과 집합을 출력하는 SQL문은 무엇인가?

```
DROP TABLE TB_SAL_TEST;

CREATE TABLE TB_SAL_TEST
(
    SAL_NO CHAR(10)
, SAL_AMT NUMBER(15)
, EMP_NO CHAR(10)
, CONSTRAINT PK_TB_SAL_TEST PRIMARY KEY(SAL_NO)
);

DROP TABLE TB_EMP_TEST;
CREATE TABLE TB_EMP_TEST
(
    EMP_NO CHAR(10)
, EMP_NM VARCHAR2(50)
, CONSTRAINT PK_TB_EMP_TEST PRIMARY KEY(EMP_NO)
);

INSERT INTO TB_SAL_TEST VALUES ('1000000001', 250000, '1000000001');
INSERT INTO TB_SAL_TEST VALUES ('1000000002', 250000, '1000000001');
INSERT INTO TB_SAL_TEST VALUES ('1000000003', 250000, '1000000001');
INSERT INTO TB_SAL_TEST VALUES ('1000000004', 250000, '1000000003');

INSERT INTO TB_EMP_TEST VALUES ('1000000001', '이경오');
INSERT INTO TB_EMP_TEST VALUES ('1000000002', '이말년');

COMMIT;
```

```
SELECT A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM TB_SAL_TEST A
WHERE NOT EXISTS
( SELECT 1
  FROM TB_EMP_TEST K
  WHERE K.EMP_NO = A.EMP_NO
)
```

SAL_NO	SAL_AMT	EMP_NO
1000000004	250000	1000000003

문제 23. 아래와 같이 테이블을 생성하고 데이터를 입력하였다. 문제의 SQL문과 동일한 결과 집합을 출력하는 SQL문은 무엇인가?

① **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE B.EMP_NO **IS NULL**

② **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE B.EMP_NO **IS NOT NULL**

③ **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE A.EMP_NO **IS NULL**

④ **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE A.EMP_NO **IS NOT NULL**

문제 23. 아래와 같이 테이블을 생성하고 데이터를 입력하였다. 문제의 SQL문과 동일한 결과 집합을 출력하는 SQL문은 무엇인가?

① **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE B.EMP_NO **IS** NULL

② **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE B.EMP_NO **IS** NOT NULL

③ **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE A.EMP_NO **IS** NULL

④ **SELECT** A.SAL_NO, A.SAL_AMT, A.EMP_NO
FROM
 TB_SAL_TEST A **LEFT OUTER JOIN** TB_EMP_TEST B
 ON (A.EMP_NO = B.EMP_NO)
WHERE A.EMP_NO **IS** NOT NULL

문제 24. 아래의 계층 형 쿼리에서 리프 데이터면 1, 그렇지 않으면 0을 출력하고 싶을 때 사용하는 키워드로 알맞은 것은?

```
SELECT LEVEL LVL
, LPAD(' ', 4*(LEVEL-1)) || A.EMP_NO
, A.DIRECT_MANAGER_EMP_NO
, ①
FROM TB_EMP A
START WITH A.DIRECT_MANAGER_EMP_NO IS NULL
CONNECT BY PRIOR A.EMP_NO = A.DIRECT_MANAGER_EMP_NO
;
```

- ① ISLEAF_CONNECT_BY
- ② IS_LEAF_CONNECT_BY
- ③ CONNECT_BY_LEAF
- ④ CONNECT_BY_ISLEAF

문제 24. 아래의 계층 형 쿼리에서 리프 데이터면 1, 그렇지 않으면 0을 출력하고 싶을 때 사용하는 키워드로 알맞은 것은?

```
SELECT LEVEL LVL
, LPAD(' ', 4*(LEVEL-1)) || A.EMP_NO
, A.DIRECT_MANAGER_EMP_NO
, ①
FROM TB_EMP A
START WITH A.DIRECT_MANAGER_EMP_NO IS NULL
CONNECT BY PRIOR A.EMP_NO = A.DIRECT_MANAGER_EMP_NO
;
```

- ① ISLEAF_CONNECT_BY
- ② IS_LEAF_CONNECT_BY
- ③ CONNECT_BY_LEAF
- ④ **CONNECT_BY_ISLEAF**

문제 25. 아래와 같은 테이블이 있다. SQL문의 결과로 올바른 것은?

```
CREATE TABLE TB_SUM_TEST_1
(
    KEY1 CHAR(1)
, VAL1 NUMBER(15)
);

CREATE TABLE TB_SUM_TEST_2
(
    KEY2 CHAR(1)
, VAL2 NUMBER(15)
);

INSERT INTO TB_SUM_TEST_1 VALUES ('A', 1);
INSERT INTO TB_SUM_TEST_1 VALUES (NULL, 2);
INSERT INTO TB_SUM_TEST_1 VALUES ('B', 3);
INSERT INTO TB_SUM_TEST_1 VALUES ('C', 4);

COMMIT;

INSERT INTO TB_SUM_TEST_2 VALUES ('A', 1);
INSERT INTO TB_SUM_TEST_2 VALUES (NULL, 2);
INSERT INTO TB_SUM_TEST_2 VALUES ('B', 3);

COMMIT;
```

SELECT * FROM TB_SUM_TEST_1;

KEY1	VAL1
----	----
A	1
NULL	2
B	3
C	4

SELECT * FROM TB_SUM_TEST_2;

KEY2	VAL2
----	----
A	1
NULL	2
B	3

```
SELECT *
FROM TB_SUM_TEST_1 A
, TB_SUM_TEST_2 B
WHERE A.KEY1 <> B.KEY2
ORDER BY A.KEY1, B.KEY2
;
```

①

KEY1	VAL1	KEY2	VAL2
----	----	----	----
C	4	B	3

②

KEY1	VAL1	KEY2	VAL2
----	----	----	----
A	1	B	3
B	3	A	1
C	4	A	1
C	4	B	3

③

KEY1	VAL1	KEY2	VAL2
----	----	----	----
A	1	B	3
B	3	A	1

④

KEY1	VAL1	KEY2	VAL2
----	----	----	----
C	4	A	1
C	4	B	3

문제 25. 아래와 같은 테이블이 있다. SQL문의 결과로 올바른 것은?

```
CREATE TABLE TB_SUM_TEST_1
(
    KEY1 CHAR(1)
, VAL1 NUMBER(15)
);

CREATE TABLE TB_SUM_TEST_2
(
    KEY2 CHAR(1)
, VAL2 NUMBER(15)
);

INSERT INTO TB_SUM_TEST_1 VALUES ('A', 1);
INSERT INTO TB_SUM_TEST_1 VALUES (NULL, 2);
INSERT INTO TB_SUM_TEST_1 VALUES ('B', 3);
INSERT INTO TB_SUM_TEST_1 VALUES ('C', 4);

COMMIT;

INSERT INTO TB_SUM_TEST_2 VALUES ('A', 1);
INSERT INTO TB_SUM_TEST_2 VALUES (NULL, 2);
INSERT INTO TB_SUM_TEST_2 VALUES ('B', 3);

COMMIT;
```

SELECT * FROM TB_SUM_TEST_1;

KEY1	VAL1
----	----
A	1
NULL	2
B	3
C	4

SELECT * FROM TB_SUM_TEST_2;

KEY2	VAL2
----	----
A	1
NULL	2
B	3

```
SELECT *
FROM TB_SUM_TEST_1 A
, TB_SUM_TEST_2 B
WHERE A.KEY1 <> B.KEY2
ORDER BY A.KEY1, B.KEY2
;
```

①

KEY1	VAL1	KEY2	VAL2
----	----	----	----
C	4	B	3

②

KEY1	VAL1	KEY2	VAL2
----	----	----	----
A	1	B	3
B	3	A	1
C	4	A	1
C	4	B	3

③

KEY1	VAL1	KEY2	VAL2
----	----	----	----
A	1	B	3
B	3	A	1

④

KEY1	VAL1	KEY2	VAL2
----	----	----	----
C	4	A	1
C	4	B	3